

Parallel Algorithms

Sylvain Contassot-Vivier

Université de Lorraine, LORIA, France

Introduction

Introduction

Computers allow for the efficient automatic resolution of many problems

However, expectations are endlessly increasing according to the complexity and size of the addressed problems !

⇒ Early apparition of the concept of *parallelism* :

- Executing several tasks *simultaneously*
- Possible at several levels :
 - **Processor** : computing units, pipelines, multiple cores
 - **Machine** : parallel machines and multi-processors stations (SMP)
 - **Beyond single machine** : clusters of machines (COW/NOW), grids
- *Double interest* :
 - Increase of the *computing power* (reduced execution time)
 - Increase of the *memory capacity* (increased problem size)

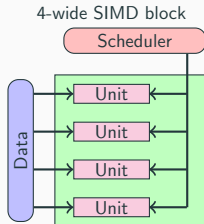
- **Parallel models and parallel systems**
 - Classification of Flynn and parallel systems
- **Assessment of parallel algorithms**
 - Speed Up, work, efficiency,...
- **Parallel programming models**
 - Data parallelism, control parallelism, flow parallelism
- **Design of parallel algorithms**
 - Shared memory systems programming
 - Distributed memory systems programming
- **Load balancing**

Parallel models

Many classifications are possible according to the chosen criteria

Flynn's classification is the most commonly used
(nb instructions / nb data) :

- *SISD* (Single Instruction Single Data) :
 - Sequential scalar processor systems
- *SIMD* (Single Instruction Multiple Data) :
 - Vector processor systems : vector units
 - Large number of small computing units working in parallel
- *SIMT* (Single Instruction Multiple Threads) :
 - Extension of SIMD : set of SIMD units
 - Every thread has its own program counter
- *MISD* (Multiple Instructions Single Data) :
 - No/few implementations, application field too restricted
- *MIMD* (Multiple Instructions Multiple Data) :
 - *SPMD* (Single Program Multiple Data) : the most common scheme
 - *MPMD* (Multiple Programs Multiple Data) : collaborative algorithms



Parallel systems

Classification based on physical size :

- **Mono-processor machines :**
 - Workstations, classical PCs but also vector processor machines
- **Parallel machines :**
 - Shared or distributed memory multi-processors machines
- **Local clusters :**
 - Independent machines connected via a LAN
- **Distributed clusters / grids :**
 - Machines of any previous types interconnected via WAN (Internet)

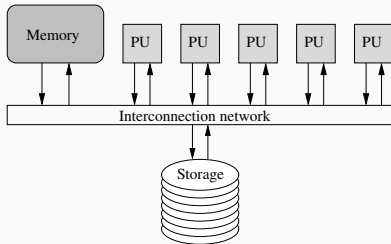
The *memory organization* is essential :

- *Shared* or *Distributed*
⇒ Directly influences the programming scheme

Shared memory systems

Architecture :

- Computing units connected to a *common central memory*



Advantages :

- No need for data distribution
- Data exchanges between units via the memory
⇒ Implicit and fast !

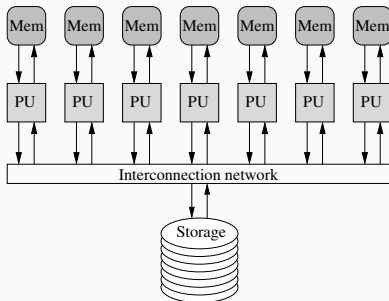
Disadvantages :

- Need for high bandwidth bus and memory
- Management of *concurrent accesses* (*mutual exclusion*)

Distributed memory systems

Architecture :

- Computing units owning a *local memory*



Advantages :

- No concurrent memory accesses
- Bus and memory bandwidth less critical

Disadvantages :

- Need for data distribution among the units
- Use of *data transfers (messages)* between units

Local clusters

Two types :

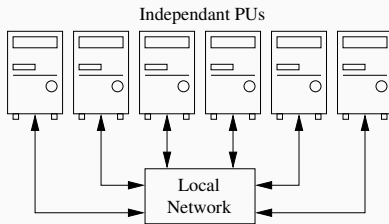
- *COWS/NOWS* :
 - Use of mass-production machines (low cost)
- *Integrated systems* :
 - Designed by major companies
 - Specific hardware : racks, optimized network (higher cost)
 - Optimized software environment (system, programming tools,...)

Advantages :

- Configuration flexibility
- Ease of maintenance

Disadvantages :

- Relative slowness of the network
(tends to decrease)



Grids

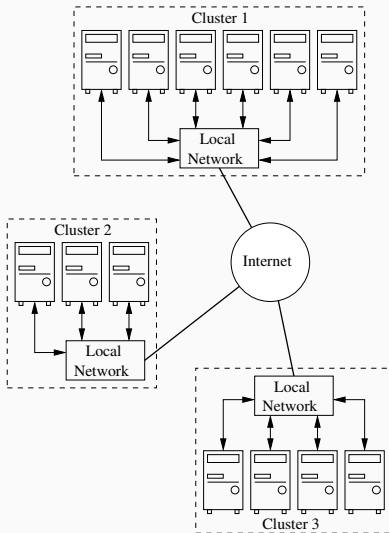
Interconnexion of geographically distant systems via the global network

Advantages :

- Potential computing power and memory amount much more important

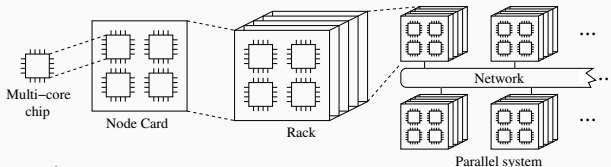
Disadvantages :

- Larger impact of the network
- More complex resources management
- *Reliability*
- *Security*
- ...



Overview of HPC systems

Parallel systems are more and more *hierarchical* with several levels of parallelism



Typically, we have :

- A set of interconnected machines containing :
 - several processors :
 - multi-cores with internal pipelines
 - and potentially some hardware accelerators :
 - GPU, FPGA,...

Efficient use must take into account *all* levels and their *specificities* :

- Shared or distributed memory, SIMD or MIMD architecture,...

⇒ This is the challenge of *parallel algorithms*!

Super-computers

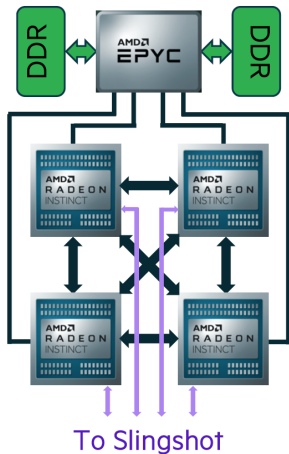
Updated list at <https://www.top500.org>

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,340,000	1,809.00	2,821.10	29,685
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	1,000.00	1,226.28	15,794

The 4 most powerful computers in 2025

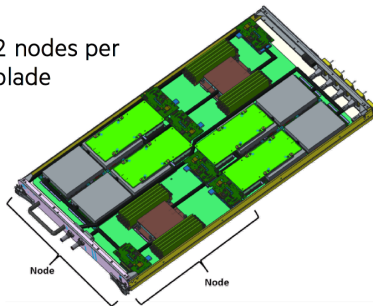
<https://www.top500.org/lists/top500/2025/11>

Detail of a Frontier node



AMD GPU
(ORNL)

2 nodes per
blade



COPYRIGHT 2020 HPE

Composition of a node of the **Frontier** super-computer

Algorithmic concepts of parallelism

Algorithmic concepts

At the algorithmic level, we consider a system with :

- an *unlimited number* of computing units (distinct IDs)
- no particular hypothesis over the architecture (context dependent)

Generic algorithmic action :

- *Spatial loop* :

```
for num from beg to end do in parallel
    ... // Instructions executed by units between beg and end
endfor
```

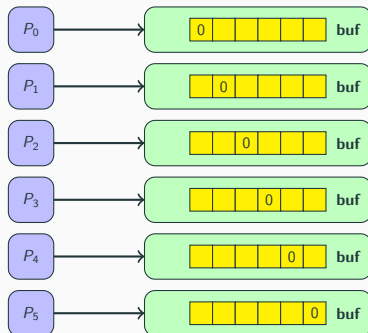
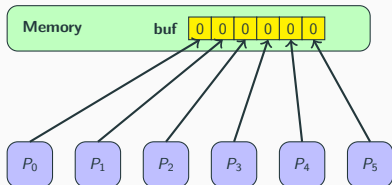
- The loop iterations are *distributed* to the units whose IDs are between beg and end
- The variable num has a different value for each unit

⚠ In *distributed memory*, the *variables* are *local* to the unit that does the action !

Example of parallel loop

```
for i from 0 to 5 do in parallel  
  buf[i] ← 0  
endfor
```

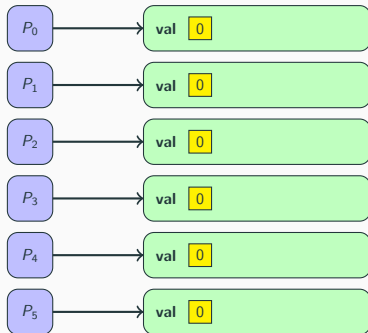
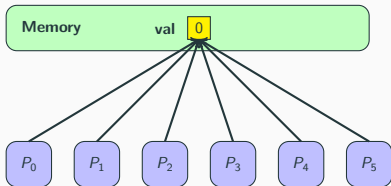
- **Shared memory** : parallel initialization of array `buf` to 0
- **Distributed memory** : each unit modifies only the cell of `buf` at the index of its ID, in its local memory



Another example of parallel loop

```
for i from 0 to 5 do in parallel  
  val ← 0  
endfor
```

- **Shared memory** : variable `val` is set to 0 (at best, the loop is useless)
- **Distributed memory** : each unit sets its local copy of `val` to 0



Distinct parallel executions and Bernstein's conditions

Distinct tasks can be executed in parallel by using a loop over the involved units and some tests over the units IDs

```
// Parallel computation of a polynomial expression in shared memory
for p from 1 to 3 do in parallel
  switch p :
    1 : res1 ← 3*z + 2           // Computation on unit 1
    2 : res2 ← x^2 + 2*x - 5     // Computation on unit 2
    3 : res3 ← 3*y^2 - 2*y + 7  // Computation on unit 3
  endswitch
endfor // Waiting for the end of all computations
// Back to sequential
res ← res1 * res2 * res3
```

It is possible to use if then else instead of switch

Parallel computations must satisfy the *Bernstein's conditions* (computational *independence*) :

- $R(C_1) \cap W(C_2) = \emptyset$ with $R(A)$ = variables read by A
- $W(C_1) \cap R(C_2) = \emptyset$ and $W(A)$ = variables modified by A
- $W(C_1) \cap W(C_2) = \emptyset$

Mutual exclusion

Sometimes, the units must execute some instructions in an *exclusive* way

Example of the summation of the array cells in shared memory :

```
sum ← 0
for i from 0 to n-1 do in parallel
  sum ← sum + buf[i] // Error → simultaneous writings in sum
endfor
```

Required algorithmic elements to manage *mutual exclusion* :

- *Atomic operation* : cannot be interrupted by any other process
 - hardware level mechanisms : Test_And_Set, Compare_And_Swap,...
- *Critical section* : *protected* block of instructions
 - Can be executed by *only one unit at a time*

⇒ Critical sections are built with atomic operations

Bakery's algorithm (Lamport) allows to implement mutex
without atomic operations when there is a *fixed number of units*

Locks for mutual exclusion

We use *locks* : shared elements with *exclusive modifications*

Initialization of a lock :

- `initLock(v)` :
 - *Initialize* the lock v in the unlocked state
 - Must be executed by *a single unit*

We define two *atomic operations* to modify locks :

- `lock(v)` :
 - *Waits* while v is locked
 - *Locks* the access to v as soon as it is possible
- `unlock(v)` :
 - *Unlocks* the access to v
 - Must be executed by the unit that has initially locked v

A classical *checking* operation can be added :

- `isLocked(v)` :
 - Returns True if v is locked
 - Returns immediately (*non blocking* function)

Back to the summation of array cells

Summation in shared memory with mutual exclusion :

```
initLock(v) // Initialization of lock v
sum ← 0
for i from 0 to n-1 do in parallel
    lock(v) // Gets the lock : start of critical section
    sum ← sum + buf[i] // Exclusive writing in sum
    unlock(v) // Frees the lock : end of critical section
endfor
```

Execution :

1. All the units try to lock v
2. *Only one* succeeds → the others wait
3. At the end of the critical section, the unit that has the lock frees it
4. Another unit takes it, and so on...
5. ...until all the units have executed the critical section

⚠ *Deadlocks* are possible when locks are not used properly!! ⚠

⚠ *Sequentialization* effect is possible if not efficiently used!! ⚠

When making use of locks ?

In many cases locks can be avoided :

- Reordering the computations to extract parallelism without locks

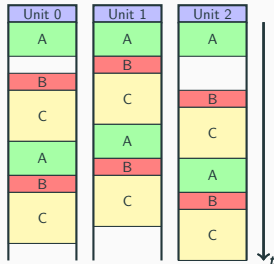
Sometimes, locks are mandatory ☹

→ Reducing as most as possible the exclusive part

- The induced sequentiality has a limited impact on performance

Example :

```
initLock(v) // Initialisation of the lock
for i from 0 to 2 do in parallel
  ... A ... // Computation before the critical section
  lock(v)   // Getting the lock : start of the critical section
  ... B ... // Exclusive instructions (critical section)
  unlock(v) // Freeing the lock : end of the critical section
  ... C ... // Computation after the critical section
endfor
```



Performance assessment of parallelism

Performance assessment of parallelism

For a given parallel algorithm, we consider :

- $T_1(n)$ = time to solve a problem of size n with 1 processing unit
- $T_p(n)$ = time to solve the same problem with p units
→ This is the time of the *last* unit to finish !

Speedup :

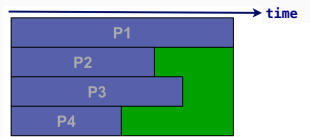
$$S_p(n) = \frac{T_1(n)}{T_p(n)}$$

- Maximum value is p (problem totally parallel)

Work :

$$W_p(n) = p \cdot T_p(n)$$

- Represents the *computing resources* required by the algorithm
- Rarely the same as the actual use of the resources

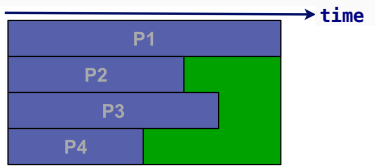


Performance assessment of parallelism

Efficiency :

$$E_p(n) = \frac{T_1(n)}{W_p(n)} = \frac{S_p(n)}{p}$$

- Represents the ratio of resources *actually used* by the algorithm
- Ratio of the used part (blue) over the total surface of the rectangle



- Value between 0 and 1 : 1 = 100% efficiency
- The objective is thus to *minimize the unused part* (green)

Remarks

⚠ A good speedup is not synonym of efficiency !

Example : comparison of two parallel executions

- $T_1(n) = 100$
- $T_4(n) = 50 \Rightarrow S_4(n) = 2 \Rightarrow W_4(n) = 200 \Rightarrow E_4(n) = 0,5$
- $T_8(n) = 33 \Rightarrow S_8(n) = 3 \Rightarrow W_8(n) = 264 \Rightarrow E_8(n) = 0,38$

S and E depends on n but also on p !

Sometimes, performance results can be *super-linear* :

- This comes generally from *cache memory* problems
- Problem too large to be totally stored in RAM with few processors
- The use of external caches slows down the execution of the algorithm
- Induces an additional speedup when external caches are no longer used (when the number of units is large enough)
- More rarely, there might be *instruction saving* when passing from sequential to parallel algorithm (especially with vector computing)

Example of instruction saving

Sequential algorithm :

```
for i from 0 to 99 do // test of index = 100 ops
  buf[i] ← 0          // array cell assignment = 100 ops
endfor                // index increment = 100 ops
```

Vectorized algorithm (shared memory) with 100 units :

```
for i from 0 to 99 do in parallel // index computation = 1 op
  buf[i] ← 0                      // array cell assignment = 1 op
endfor                            // no operation = 0 op
```

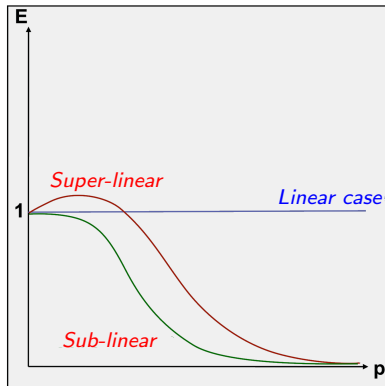
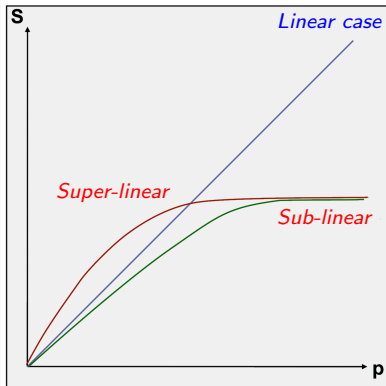
We have :

- $T_1(A) = 300$ and $T_{100}(A) = 2$

$$\Rightarrow S_{100}(A) = \frac{300}{2} = 150$$

$$\Rightarrow E_{100}(A) = \frac{150}{100} = 1,5 > 1 \Rightarrow \text{super-linear !}$$

Typical performance curves for linear algorithms



A non-linear example of super-linearity

Context :

- Problem of size n and an algorithm with sequential time $\mathcal{O}(n^2)$
- P computing units (much smaller than n)
- Decomposition into P subparts of size $\frac{n}{P}$
 $\Rightarrow$ Each part is processed in time $\mathcal{O}\left(\frac{n^2}{P^2}\right)$
- The merging of partial results is assumed to be in time $\mathcal{O}(P.n)$

We obtain $T_1(n) = n^2$ and $T_P(n) = \frac{n^2}{P^2} + P.n = \left(\frac{n}{P^2} + P\right).n$:

$$\Rightarrow S_P(n) = \frac{n}{\frac{n}{P^2} + P} \rightarrow P^2 \text{ when } n \gg P \quad (\rightarrow 1 \text{ when } P \approx n)$$

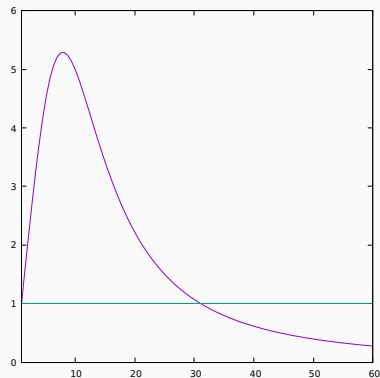
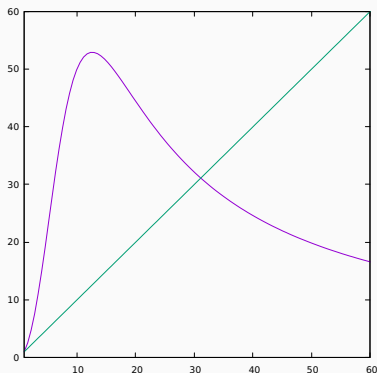
$$\Rightarrow E_P(n) \rightarrow \frac{P^2}{P} = P \Rightarrow \text{super-linear !}$$

To obtain a speedup of P , the constraint is $P = \frac{n}{\frac{n}{P^2} + P}$

$$\Rightarrow \frac{n}{P} + P^2 = n \quad \Rightarrow \quad n + P^3 = nP \quad \Rightarrow \quad \frac{P^3}{P-1} = n$$

We deduce that P must be around $\sqrt{n}$

Curves of the quadratic example



Results for $n=1000$:

- Speedup of 52.86 and efficiency of 4.06 with 13 units
- Speedup of 42.32 and efficiency of 5.29 with 8 units
- Speedup of 31.21 and efficiency of 1.01 with 31 units ($\approx \sqrt{1000}$)

Discussion over super-linearity

Indeed, it depends on what is evaluated :

- *Sequential vs Parallel* : best algorithm from each context
 - Explicitly identify the algorithmic/behavioral differences
⇒ Super-linearity is possible
- *Parallel algorithm vs itself* : inner quality
 - Compare the *parallel execution* time with p units to its *sequential simulation* time with 1 unit
 - Without considering memory limitations, the simulation time will be at most p times the parallel execution time
⇒ Maximal speedup is p
 - Previous example with vectorized algorithm :
Sequential simulation of vectorized program = 200 ops
Speedup is $\frac{200}{2} = 100$ and the efficiency is 1 (100%)
 - Comparison of execution times of the *same* algo with 1 and p units :
⇒ *Amdahl's law*

Amdahl's law

Generally, two distinct parts can be identified in an algorithm :

- A *purely sequential* part (no parallelism is possible) :
 - It is associated a ratio (percentage) of the overall algorithm :
 $0 \leq R_s(n) \leq 1$
- A *parallelizable* part (parallelism can be applied) :
 - Its ratio is $1 - R_s(n)$
(complementary of the sequential part)

So, for a problem of size n , we have :

- $T_s(n) = R_s(n) \cdot T_1(n)$ = time of purely sequential part
- $T_{//}(n) = (1 - R_s(n)) \cdot T_1(n)$ = time of parallelizable part
- $T_1(n) = T_s(n) + T_{//}(n) = R_s(n) \cdot T_1(n) + (1 - R_s(n)) \cdot T_1(n)$

And the theoretically *minimal execution time* with p units is :

$$T_p(n) = T_s(n) + \frac{T_{//}(n)}{p} = \left(R_s(n) + \frac{1 - R_s(n)}{p} \right) \times T_1(n)$$

Amdahl's law

We apply it to speedup and efficiency :

$$S_p(n) = \frac{T_1(n)}{\left(R_s(n) + \frac{1-R_s(n)}{p}\right) \times T_1(n)} = \frac{1}{R_s(n) + \frac{1-R_s(n)}{p}}$$

$$E_p(n) = \frac{1}{p \times \left(R_s(n) + \frac{1-R_s(n)}{p}\right)} = \frac{1}{1 + (p-1) \times R_s(n)}$$

We obtain the following limits :

$$\lim_{p \rightarrow \infty} S_p(n) = \frac{1}{R_s(n)} \Rightarrow \text{for } R_s(n) = 10\%, \text{ speedup cannot exceed } 10$$

$$\lim_{p \rightarrow \infty} E_p(n) = 0 \mid 1 \Rightarrow \text{efficiency tends to } 0 \text{ as soon as there is a sequential part (always)}$$

Brent's principle

Extension of the sequential simulation (1 unit) to p units :

- Simulation with p units of a parallel algorithm using an undefined number of units :
 - Algorithm A with sequential cost C_1 and parallel time T_∞
 - It is possible to simulate A with p identical units in :

$$T_p = \mathcal{O}\left(\frac{C_1}{p} + T_\infty\right)$$

- How ?
 - At each time i , A executes $C_1(i)$ operations : $C_1 = \sum_{i=1}^{T_\infty} C_1(i)$
 - Each time is simulated with p units in $\left\lceil \frac{C_1(i)}{p} \right\rceil \leq \frac{C_1(i)}{p} + 1$
 - The summation over time gives the result

⇒ *Performance prediction* when the number of units is reduced

Grain et degree of parallelism

Grain :

- Size of the elementary tasks considered in the parallel process
- Choice linked to the target architecture :
 - Scale : bit, operator, expression, instruction, function, program
 - *Fine grain* : generally SIMD in share memory
 - *Coarse grain* : generally MIMD in distributed memory
- Several grain sizes are possible within a same algorithm (hierarchy)

Degree :

- Gives the number of possible simultaneous operations (≥ 1)
 - May vary during the algorithm
(notion of distinct parts with different degrees)
 - The parts of degree 1 are purely sequential
- ⇒ Notions of minimal, maximal and average degree
- Allows to evaluate the performance for a given number of units
 - Provides information over the number of units to use

Exploitation of the degree

Let's consider a parallel program composed of n parts with respective degrees d_i and execution times t_i such that : $T_1 = \sum_{i=1}^n d_i t_i$

We have :

$$S_p = \frac{\sum_{i=1}^n d_i t_i}{\sum_{i=1}^n \left\lceil \frac{d_i}{p} \right\rceil t_i}$$

When $p \geq \max_i d_i$ the maximal speedup is $S_p = \frac{\sum_{i=1}^n d_i t_i}{\sum_{i=1}^n t_i}$

However, this does not necessarily correspond to the *maximal efficiency*

Example : program with 3 parts

d_i	2	7	3
t_i	5	20	7

What is the number of units giving the best efficiency ?

Exploitation of the degree

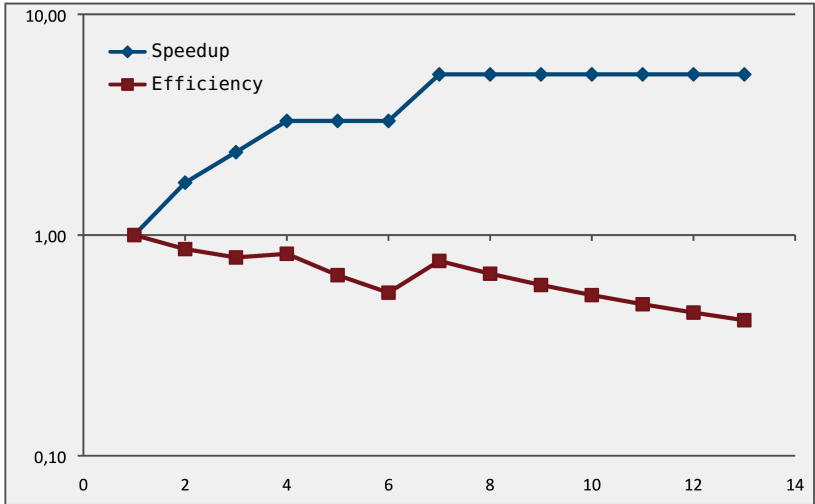
Example : program with 3 parts

d_i	2	7	3
t_i	5	20	7

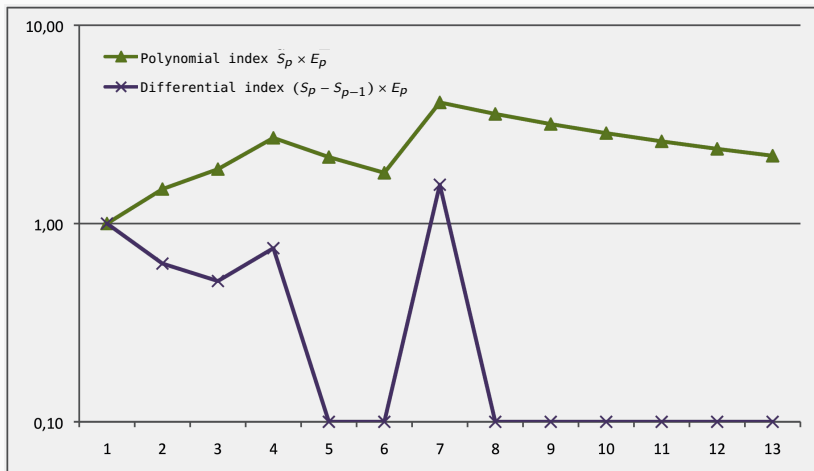
Performance in function of the number of units :

Nb units	1	2	3	4	5	6	7	8	9
Time	171	99	72	52	52	52	32	32	32
Speedup	1	1,73	2,38	3,29	3,29	3,29	5,34	5,34	5,34
Efficiency	1	0,86	0,79	0,82	0,66	0,55	0,76	0,67	0,59

Performance plots



Global indices



Parallel programming models

Parallel programming models

Data parallelism :

- Data distribution over the parallel system
- Generally based on the data regularity
- Application of a same computation to different data

Control/Task parallelism :

- Calculations distribution over the system
- Each unit computes a part of the global result
- Data may be duplicated

Pipeline parallelism :

- Task decomposition into successive sub-tasks : assembly-line work
- Tasks must have similar execution times

Mixing of previous schemes :

- Distribution of data and computations over the system
- Generally more complex management (additional processes)

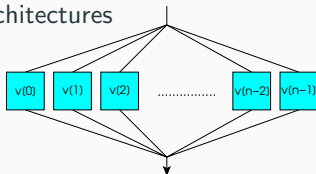
Comparative Example

Computation of n values of a given polynomial :

```
for i from 0 to n-1 do
  v[i] ← a + b.x[i] + c.x[i]^2 + d.x[i]^3 + e.x[i]^4 + f.x[i]^5
endfor
```

Data parallelism : mostly used with *SIMD* architectures

```
for i from 0 to n-1 do in parallel
  v[i] ← a + b.x[i] + c.x[i]^2 + d.x[i]^3
        + e.x[i]^4 + f.x[i]^5
endfor
```



- Requires at most n units : beyond that, it is useless !
- When there are less than n units, each one computes several $v[i]$

⇒ Assignment of data subsets to each unit :

Example for $n = 20$ and $p = 3$, computations can be distributed as :

- P0 : v[0] to v[6] (7 elements)
- P1 : v[7] to v[13] (7 elements)
- P2 : v[14] to v[19] (6 elements)

if $n \% p \neq 0$, each unit computes

$\left\lceil \frac{n}{p} \right\rceil$ or $\left\lfloor \frac{n}{p} \right\rfloor$ elements

Comparative Example

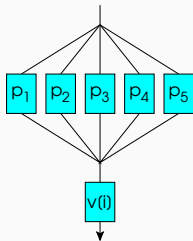
Task Parallelism : mostly used with *MIMD* machines

- Rewriting of the polynomial in a decomposed form :

$$v[i] = (a + b.x[i]) + c.x[i]^2 + x[i]^3 . ((d + e.x[i]) + (f.x[i]^2))$$

- Leads to the following algorithm :

```
for i from 0 to n-1 do
  for p from 1 to 5 do in parallel
    switch p :
      1 : p1 ← a + b.x[i]
      2 : p2 ← c.x[i]^2
      3 : p3 ← x[i]^3
      4 : p4 ← d + e.x[i]
      5 : p5 ← f.x[i]^2
    endswitch
  endfor
  v[i] ← p1 + p2 + p3.(p4 + p5)
endfor
```



- Parallelism limited to 5 units in this case
- Often necessary to merge partial results $\Rightarrow$ *synchronization !*
 - Important to have tasks with *similar execution times*

Comparative example

Pipeline parallelism : mostly used at hardware level

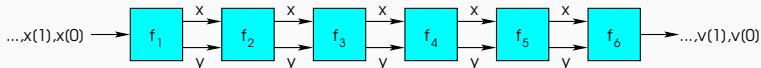
- Rewriting in the form of a *functions composition* :

$$f_1(x) = (x, f + x.0), f_2(x, y) = (x, e + x.y), f_3(x, y) = (x, d + x.y),$$

$$f_4(x, y) = (x, c + x.y), f_5(x, y) = (x, b + x.y), f_6(x, y) = a + x.y$$

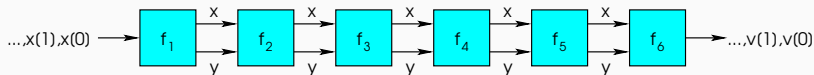
- Leads to the following computation for each $v[i]$:

$$v[i] \leftarrow f_6(f_5(f_4(f_3(f_2(f_1(x[i]))))))$$



- Each function is a *stage* of the pipeline
- Several execution phases :
 - *Filling* : the first elements of the data stream enter the pipeline
 - *Full capacity* : all the pipeline stages are active
 - *Emptying* : last elements of the data stream go through the pipeline
- Parallel degree given by the number of stages
- Maximal efficiency when the f_i have the *same execution times*

Execution example with 8 data



⚠ The pipeline *does not reduce* the compute time of one element !

It decreases the *latency* between the results
→ *Increases the throughput* of operations !

Filling

Full run

Emptying

x[0]						
x[1]	x[0]					
x[2]	x[1]	x[0]				
x[3]	x[2]	x[1]	x[0]			
x[4]	x[3]	x[2]	x[1]	x[0]		
x[5]	x[4]	x[3]	x[2]	x[1]	x[0]	
x[6]	x[5]	x[4]	x[3]	x[2]	x[1]	v[0]
x[7]	x[6]	x[5]	x[4]	x[3]	x[2]	v[1]
	x[7]	x[6]	x[5]	x[4]	x[3]	v[2]
		x[7]	x[6]	x[5]	x[4]	v[3]
			x[7]	x[6]	x[5]	v[4]
				x[7]	x[6]	v[5]
					x[7]	v[6]
						v[7]

Data parallelism

Parallel operators that satisfy the *Bernstein's conditions*

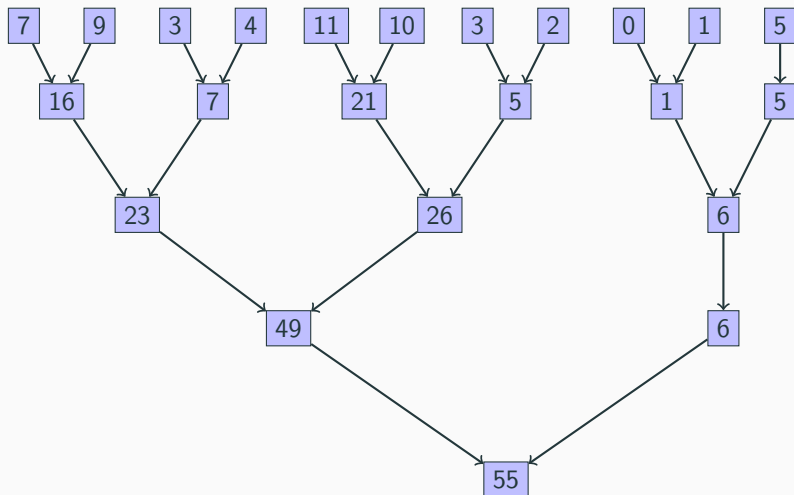
α -notation : $\alpha(\text{operator}, v1, v2)$

- k-ary operator (k vectors) executed in parallel element-wise
 - Ideally, one unit computes one element of the resulting vector
- $\Rightarrow \mathcal{O}\left(\left\lceil \frac{n}{p} \right\rceil\right)$ for vectors size n and p units

Logarithmic β -reduction : $\beta(\text{operator}, v)$

- Comes from β -reduction :
 - Binary function successively applied to the elements of a vector
 - Not directly parallel but can be useful for associative functions
 - Succession of gatherings in distinct couples : *binary tree*
 - At each step, reduction by factor 2 of the remaining data to process
- $\Rightarrow \mathcal{O}(\log_2(n))$ steps to process n data

Summation example



Brent's principle applied to the reduction

- Sequential cost : $\mathcal{O}(n)$
 - Parallel time : $\mathcal{O}(\log(n))$
 - Number of units : $\mathcal{O}(n)$
 - Same parallel time with less than $\mathcal{O}(n)$ units?
 - With p units, we have $T_p = \mathcal{O}\left(\frac{n}{p} + \log(n)\right)$
 - Choosing $p = \frac{n}{\log(n)}$ leads to $T_p = \mathcal{O}(\log(n))$
 - We can *preserve the complexity* while using *less units*!!
- ⇒ Improves the parallel efficiency (when not already maximal)

Exercise

Matrices multiplication : $A(m, l) \times B(l, n)$

```
for i from 0 to m-1 do
  for j from 0 to n-1 do
    C[i][j] ← 0
    for k from 0 to l-1 do
      C[i][j] ← C[i][j] + A[i][k] * B[k][j]
    endfor
  endfor
endfor
```

Parallel version using the functions `lin(M, num)` and `col(M, num)`

```
for i from 0 to m-1 do in parallel
  for j from 0 to n-1 do in parallel
    C[i][j] ←  $\beta(+, \alpha(\times, \text{lin}(A, i), \text{col}(B, j)))$ 
  endfor
endfor
```

Performance study

Complexities :

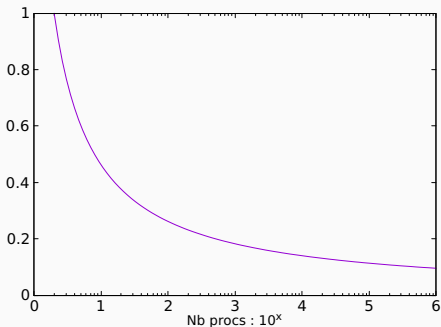
- Sequential version : $m.n.l$ ($\times$) + $m.n.l$ (+)
 - Parallel version : 1 ($\times$) + $\lceil \log_2(l) \rceil$ (+)
- $\Rightarrow$ Speedup above $m.n!$

Surface :

- Sequential version : 1
- Parallel version : $m.n.l$

If $\times \approx +$ then efficiency : $\frac{2}{\lceil \log_2(l) \rceil + 1}$

$\rightarrow$ *Efficiency sharply decreases*
when l increases !



Brent's principle application

By applying the Brent's principle, we can use *less units* :

- Surface : $m.n. \frac{l}{\lceil \log_2(l) \rceil} = \frac{m.n.l}{\lceil \log_2(l) \rceil}$
- Time : $\frac{l}{\lceil \log_2(l) \rceil} + \lceil \log_2(l) \rceil \approx \lceil \log_2(l) \rceil + \lceil \log_2(l) \rceil = 2. \lceil \log_2(l) \rceil$
- Speedup : $\frac{2.m.n.l}{2. \lceil \log_2(l) \rceil} = \frac{m.n.l}{\lceil \log_2(l) \rceil}$
- Efficiency : $\frac{m.n.l}{\lceil \log_2(l) \rceil} \times \frac{\lceil \log_2(l) \rceil}{m.n.l} = 1$

⇒ We obtain a *maximal efficiency* by using $\frac{l}{\lceil \log_2(l) \rceil}$ units !

Data distribution

Data distribution corresponds to the assignment of the data to the units

In *shared memory* :

- A datum can be read by any unit
- A datum must be modified by only one unit !

⇒ The distribution specifies which unit can modify which datum

In *distributed memory* :

- Data may not be entirely stored in the memory of each node

⇒ The distribution specifies in which node each datum is stored

Classical distributions

For n data and P units :

- *Block* :

- If $n \% P = 0$ then datum i is in block b if $b \cdot \frac{n}{P} \leq i \leq (b+1) \cdot \frac{n}{P}$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----

- Else, *remaining* data must be distributed over the first or last units

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	imbalance balance		
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16			

- *Cyclic* :

- Datum i is in block b if $i \% P = b$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----

⇒ Implicit balancing !

- *Block-cyclic* :

- Datum i is in block b of size k if $\lfloor \frac{i}{k} \rfloor \% P = b$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----

- Possible imbalance according to the choice of k !

Specific distributions

Imposed by the *dependencies* between data

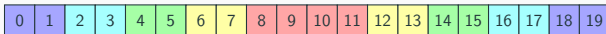
Example :

```
for i from 0 to n-1 do
  a[i] ← f(i)           // synchronization required
  b[i] ← a[i] + a[n-1-i] // between the two computations
endfor
```

⇒ Essential to *minimize units dependencies* to avoid synchronizations
(units that use data modified by other units)

Distribution is expressed by a *data selection* in function of unit number

```
for p from 0 to P-1 do in parallel
  for i from p.n/(2.p) to (p+1).n/(2.p) do // Data selection loop
    a[i] ← f(i)
    a[n-1-i] ← f(n-1-i)
    b[i] ← a[i] + a[n-1-i]
    b[n-1-i] ← b[i]
  endfor
endfor
```



Summary of data parallelism

Related to *regular data structures* :

- Potentially high degree of parallelism
- Parallelism identification is generally quite simple

However :

- *Data dependencies* reduce the degree of parallelism
- The *degree* may strongly vary between the parts of an algorithm :
 - Speedup and efficiency may not match
- Important cost in *hardware resources* (units)
- Generally limited to *static data* :
 - Less adapted to the process of dynamical data

Task parallelism

The principle is to decompose the computations in *tasks* that can be done in parallel

Important criteria :

- *Grain* :
 - Depends on the target architecture
 - Must take into account the data transfers (distributed system)
 - And the times taken by the obtained tasks
- *Dependencies* between tasks : *DAG*
 - Analysis of the dependencies to deduce simultaneous tasks
 - Task distribution over the units (*placement*)
 - Starting the tasks as soon as possible (*scheduling*)
 - Taking into account the potential communications

Example

A simple way (non optimal) is to identify groups of tasks that can be done in parallel :

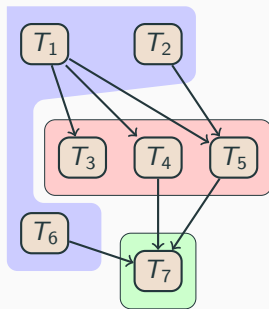
1. Tasks with no dependence
2. Tasks with satisfied dependencies

→ Order constraints between tasks groups :

- (T_1, T_2, T_6) before (T_3, T_4, T_5) before (T_7)

⇒ Good results if the tasks in a same group take similar times

⇒ Better results are possible when tasks time estimations are available



Example in function of the number of available units :

- 1 unit : $T_1, T_2, T_3, T_4, T_5, T_6, T_7$
- 2 units : $(T_1, T_2), (T_3, T_4), (T_5, T_6), (T_7)$
- 3 units : $(T_1, T_2, T_6), (T_3, T_4, T_5), (T_7)$
- 4 units : $(T_1, T_2), (T_3, T_4, T_5, T_6), (T_7)$
- 5 units and more : nothing better as there is no more than 4 independent tasks

Task Scheduling

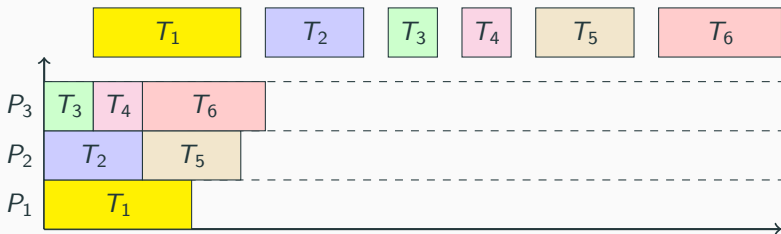
Difficult problem $\Rightarrow$ use of *heuristics*

For some problems, some heuristics produce results close to the *optimal*

Simple case : *independent* tasks with identical units

List algorithm :

- The next task is placed on the first available unit



Efficiency of the list algorithm

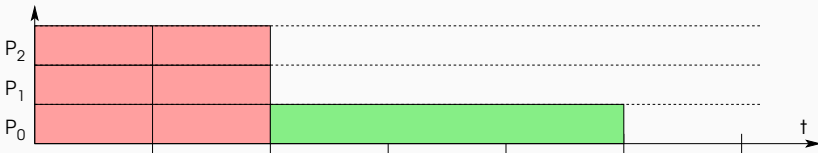
Competitiveness of a heuristic :

- Ratio between the solution produced by the heuristic and the optimal solution

Graham's Theorem (1966)

Theorem 1. *For a set of independent tasks to place on P units, a list algorithm has a competitiveness less or equal to $2 - \frac{1}{P}$.*

Find this result graphically :



Proof outline

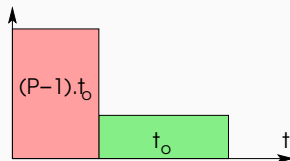
We build a *configuration* whose *optimal* solution is known :

- $n - 1$ tasks :
 - optimally scheduled over $P - 1$ units with total time t_o
 - optimally scheduled over P units with total time $\frac{(P-1) \cdot t_o}{P}$
- 1 task with execution time t_o

We deduce the *worst solution* produced by the list algorithm :

- The longest task is started last

$$\Rightarrow \text{Total time : } \frac{(P-1) \cdot t_o}{P} + t_o = \frac{(2P-1) \cdot t_o}{P}$$



The *ratio between worst and optimal cases* is : $\frac{(2P-1) \cdot t_o}{P \cdot t_o} = 2 - \frac{1}{P}$

Summary of task parallelism

Related to *independent tasks* in the computations :

- The degree of parallelism depends on this number of tasks
- More flexible approach than data parallelism :
 - Tasks can be different
 - Data can be produced dynamically

However :

- The *degree* generally stays quite limited
- The *identification* of parallelism is much less obvious

Generally used in combination with data parallelism → *mixing*

- Example of matrices multiplication :
 - *Data parallelism* used to process the set of $C[i][j]$
 - *Task parallelism* used to compute each $C[i][j]$

Pipeline parallelism

Decomposition of the computations in *successive tasks* :

- The output of a task is the input of the next task

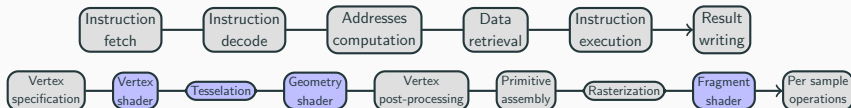
Ideal speedup for n data, k stages and time t per stage :

- Computing time starts when first data enters the pipeline
- Sequential time : $n.k.t$
- Parallel time :
 - Output time of first data = $k.t$
 - Output time of remaining data = $(n-1).t$
- Speedup :

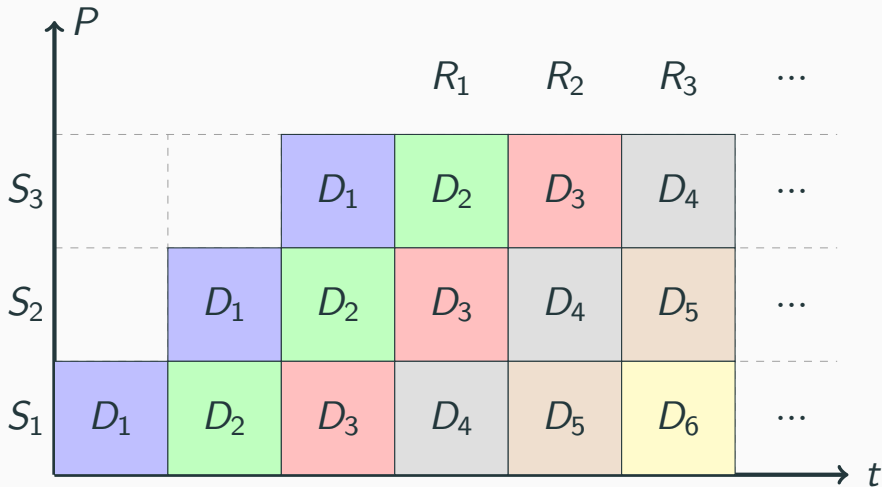
$$S_k(n) = \frac{n.k.t}{k.t + (n-1).t} = \frac{n.k}{k + n - 1} \Rightarrow \lim_{n \rightarrow \infty} S_k(n) = k$$

Main use at hardware level (circuits,...) :

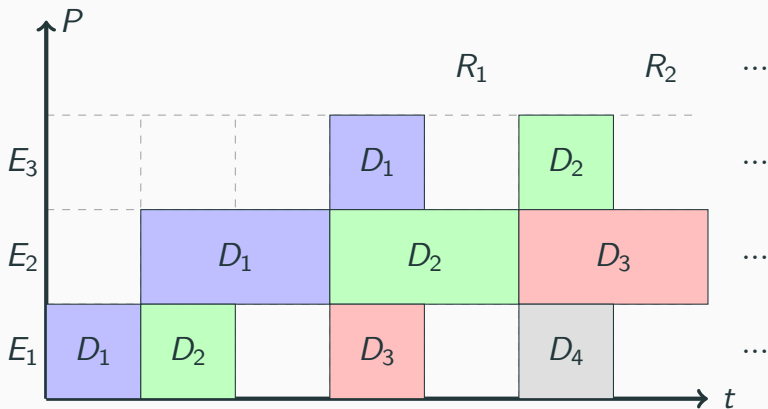
- Instruction pipeline, graphics pipeline,...



Execution diagram



Execution diagram with E_2 2 times longer than E_1 and E_3



Nonlinear pipelines

It may not be possible to decompose in stages taking the same time :

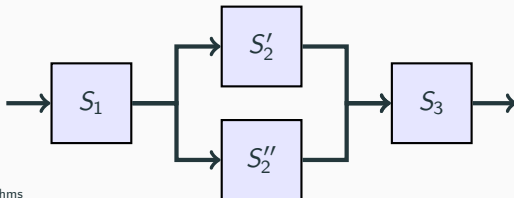
- k stages with respective execution times t_i , $1 \leq i \leq k$
- Synchronization between consecutive stages
- Output delay with full capacity : $\max_{1 \leq i \leq k} t_i$

The speedup becomes :

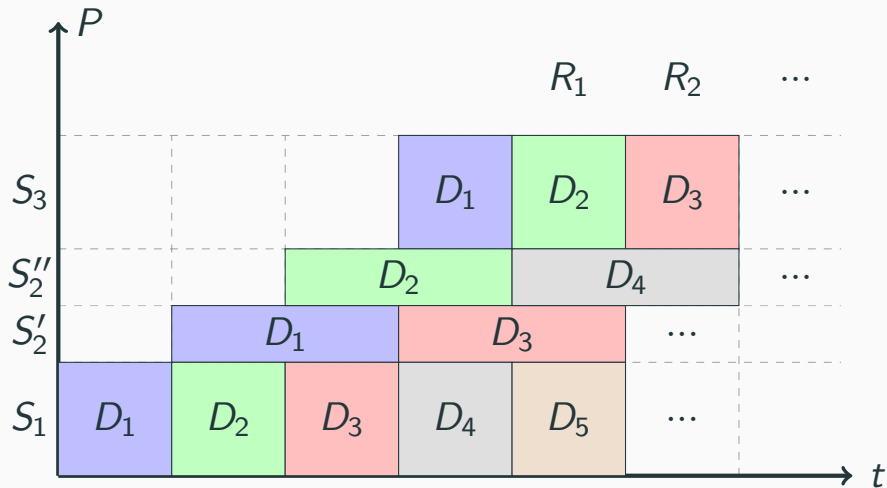
$$S_k(n) = \frac{n \sum_{i=1}^k t_i}{\sum_{i=1}^k t_i + (n-1) \cdot \max_{1 \leq i \leq k} t_i} \Rightarrow \lim_{n \rightarrow \infty} S_k(n) \leq k$$

Stages duplication :

- Example with $t_2 = 2 \cdot t_1 = 2 \cdot t_3$
- $\Rightarrow$ Duplicate the longest stages and alternate their usage



Execution diagram



Performance evaluation

With stage duplication, we obtain :

- Output time of first data = $t_1 + t_2 + t_3 = 1 + 2 + 1 = 4$
- Output time of remaining data = $(n - 1) \cdot \min(t_i) = n - 1$
- Total time = $4 + n - 1 = n + 3$

The speedup becomes :

$$S_{3+1}(n) = \frac{4n}{n+3} \Rightarrow \boxed{\lim_{n \rightarrow \infty} S_{3+1}(n) = 4} \Rightarrow \boxed{\lim_{n \rightarrow \infty} E_{3+1}(n) = 1}$$

Application to network communications

The *process* is the transfer of a message from Source to Destination passing through P successive links ($P - 1$ routers)



Decomposition of the message into packets and *immediate forwarding* of each packet on every router $\Rightarrow$ *pipeline with P stages*

Example with message length L and P links with bandwidth B :

- Without packet decomposition the message must be entirely received on each router before forwarding it $\Rightarrow$ Total delay = $\frac{P \cdot L}{B}$
- If the message is cut into k packets, we have :
 - Delivery delay of 1st packet : $\frac{P \cdot L}{k \cdot B}$
 - Delivery delay of the remaining packets : $\frac{(k-1) \cdot L}{k \cdot B}$
- $\Rightarrow$ Total delay : $\frac{(P+k-1) \cdot L}{k \cdot B}$
- When k is large enough, the delay tends towards $\frac{L}{B}$

$\Rightarrow$ *Division by P of the transfer time !*

Summary of pipeline parallelism

Related to *consecutive tasks* :

- Efficient processing of *data streams*
- Adapted to vector computing and electronic circuits

However :

- When data are entirely available
 - ⇒ Data parallelism may be better suited
- *Decomposition into tasks* taking similar times may not be possible :
 - Potential *duplication* of the longest stages

Mostly useful with :

- *Sequential* data streams
- Task taking *similar* times

Parallelism in distributed memory systems

Parallelism in distributed memory systems

Main contexts of distributed memory :

- Interconnected independent machines, super-computers

⇒ No implicit global synchronization !

Use of *explicit messages* through communications :

- *Point to point* :
 - Between two specified machines (*source*, *destination*)
 - *Synchronous* : rendez-vous (blocking) between machines
 - *Asynchronous* : no rendez-vous (blocking or non-blocking)
- *Global communications* :
 - *one-to-all* : diffusion or distribution
 - *all-to-one* : reduction or collection
 - *all-to-all* : multi-diffusion, multi-distribution,...

Point-to-point communications

Two functions :

- *Send* : `send(buffer, size, dest, tag)`
 - `buffer` : array containing the data to send
 - `size` : total size of the data to send (in information units)
 - `dest` : identifier of the destination machine
 - `tag` : integer used for filtering when receiving
- *Receive* : `recv(buffer, size, src, tag)`
 - `buffer` : array where received data are stored (*must be allocated*)
 - `size` : size of data to receive (in information units)
 - `src` : identifier of the source machine
 - `tag` : filter upon the specified tag

FIFO Communications :

- Between two machines, *order* of arrivals on destination is the *same* as the order of departures from the source

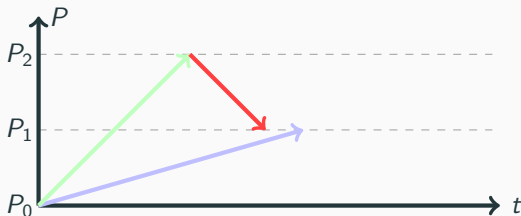
⇒ *No overtaking!*

Disorganization induced by communication delays

Communication *delays* may modify receptions order

Example :

- 3 machines : P_0 sends data to P_1 and P_2
- When P_2 receives data from P_0 , it sends some results to P_1
- When P_1 receives data from P_0 , it waits for the data from P_2



$\Rightarrow P_1$ may receive the results of P_2 *before* the data from P_0 !

How to handle receptions

Buffering of received messages :

- Source and *sequence number* are stored with each received message
- At a reception request, retrieval of the message with corresponding source and sequence number

⇒ May be *expensive in memory* when $\text{freq}_{arr} > \text{freq}_{rec}$!

- Sometimes necessary to *discard* some messages ⇒ *message loss* !

Explicit synchronizations :

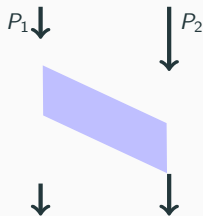
- *Synchronous* communications
- Explicit synchronization *barriers*

⇒ Generally imply a *time overhead* !

Communication modes

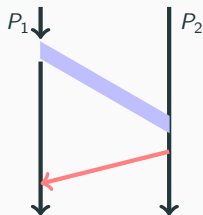
Synchronous : `ssend` et `srecv`

- Send and receive operations are *simultaneous*
 - Similar to phone call
 - Both participants must be ready to communicate (rendez-vous)
 - After sending, source is sure that destination has received the data



Asynchronous :

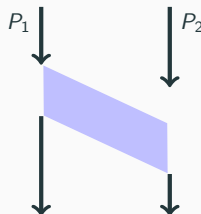
- Send and receive are *independent* in time :
 - Similar to mail (postal/electronic)
 - Potential waiting on the receiver side
 - The sender does not know when the destination receives the data
 - Acknowledgment can be used to inform the sender



Blocking and non-blocking asynchronous modes

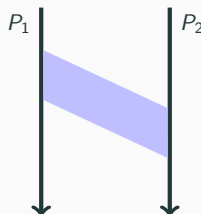
Blocking mode : `bsend` et `brecv`

- *Waiting* for the end of the communication
- The end of the sending on source side does not imply that data are received on destination side



Non-blocking mode : `nsend` et `nrecv`

- *No waiting* for the end of the communication
 - Delegation of work to the network interface
 - Required hardware capacities : computations *and* communications
- Enables the *overlapping* of computation by communication
- Management is more difficult :
 - Necessary to ensure operations *consistency*
 - Additional functions `comDone` and `comWait`



Functions for the non-blocking mode

Functions `nsend` and `nrecv` return a *request identifier*

Checking the completion of a communication : `comDone(id)`

- Returns `True` if communication `id` is finished and `False` otherwise

Waiting for communication completion : `comWait(id)`

- Blocks the process while communication `id` is not completed

Link between both modes :

```
bsend  $\equiv$  id  $\leftarrow$  nsend(...)  
comWait(id)
```

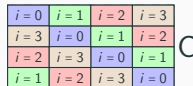
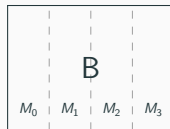
The interest is to *insert computations* between `nsend` and `comWait`

$\Rightarrow$ *Gain of time!*

Example of matrices multiplication

Computation of $C = A \times B$ with $A(m, l)$ and $B(l, n)$ over P machines :

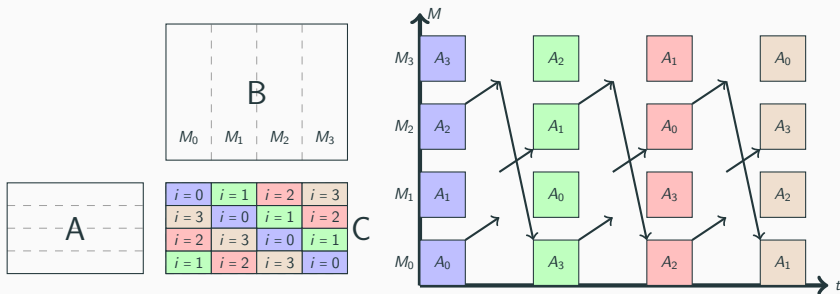
- Machines are logically organized as a *ring* : M_0 to M_{P-1}
- Decomposition of A in P horizontal strips
- Decomposition of B in P vertical strips
- Decomposition of C in $P \times P$ blocks
- Each machine k ($0 \leq k \leq P-1$) :



- Stores strip k of B in memory
 - Stores blocks in column k of C in memory
 - Starts with strip k of A in memory
- At each iteration i ($0 \leq i \leq P-1$), machine k :
 - Computes block of C associated to strips $k+i$ of A and k of B
 - Sends strip $k+i$ of A to the next machine (cyclic order)
 - Receives strip $k+i-1$ of A from previous machine (cyclic order)

$\Rightarrow P$ iterations are required to complete the computation

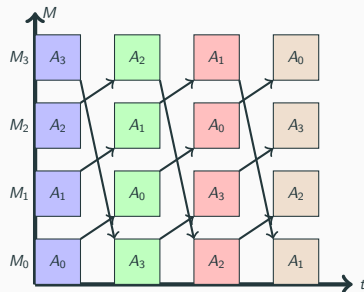
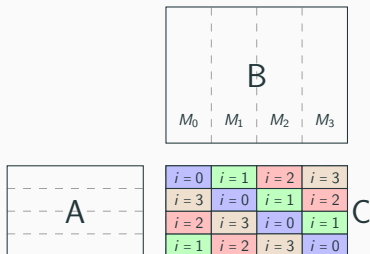
Distributed algorithm without overlapping



```

for M from 0 to P-1 do in parallel
  for i from 0 to P-2 do
    blockC ← CM[(M+P-i)%P]
    multiplication(curStripA, stripB, blockC) // Circulation loop of strips of A
    if M%2 = 0 then                          // Current block of column M of C to compute
      ssend(curStripA, (m/P)*l, (i+1)%P, 1) // C block computation with current strips of A and B
      srecv(nxtStripA, (m/P)*l, (i+P-1)%P, 1) // Organization of communications between neighbors
    else
      srecv(nxtStripA, (m/P)*l, (i+P-1)%P, 1) // Synchronous recv of next strip of A
      ssend(curStripA, (m/P)*l, (i+1)%P, 1)  // Synchronous send of local strip of A
    endif
    exchange(curStripA, nxtStripA)           // Array swap of local strips of A for next iteration
  endfor
  multiplication(curStripA, stripB, blockC) // Computation of last block of C for column M of B
endfor
  
```

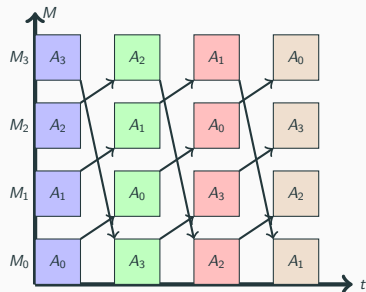
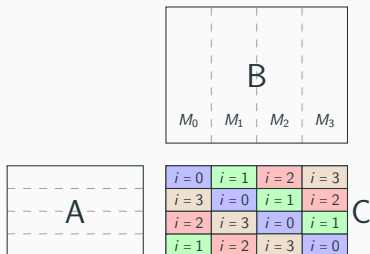
Distributed algorithm with communications overlapping



```

for M from 0 to P-1 do in parallel
  for i from 0 to P-2 do
    blockC ← CM[(M+P-i)%P]
    multiplication(curStripA, stripB, blockC) // C block computation with current strips of A and B
    //
    // Version with non-blocking sendings and blocking receptions
    //
    idS ← nsend(curStripA, (m/P)*l, (i+1)%P, 1) // Non-blocking send of local strip of A
    brecv(nxtStripA, (m/P)*l, (i+P-1)%P, 1) // Blocking rcv of the next strip of A
    comWait(idS) // Waiting of send completion
    exchange(curStripA, nxtStripA) // Array swap of local strips of A for next iteration
  endfor
  multiplication(curStripA, stripB, blockC) // Computation of last block of C for column M of B
endfor
    
```

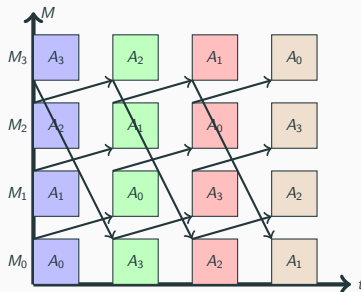
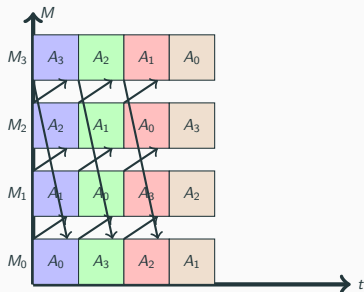

Distributed algorithm with communications overlapping



```

for M from 0 to P-1 do in parallel
  for i from 0 to P-2 do
    blockC ← CM[(M+P-i)%P]
    multiplication(curStripA, stripB, blockC) // Circulation loop of strips of A
    // Current block of column M of C to compute
    // C block computation with current strips of A and B
    //
    // Version with non-blocking receptions and blocking sendings
    //
    idR ← nrecv(nxtStripA, (m/P)*l, (i+P-1)%P, 1) // Non-blocking recv of the next strip of A
    bsend(curStripA, (m/P)*l, (i+1)%P, 1) // Blocking send of local strip of A
    comWait(idR) // Waiting of recv completion
    exchange(curStripA, nxtStripA) // Array swap of local strips of A for next iteration
  endfor
  multiplication(curStripA, stripB, blockC) // Computation of last block of C for column M of B
endfor
    
```

Distributed algorithm with comp/comm overlapping



```

for M from 0 to P-1 do in parallel
  for i from 0 to P-2 do
    blockC ← CM[(M+P-i)%P]
    idS ← nsend(curStripA, (m/P)*l, (i+1)%P, 1) // send request of local strip of A
    idR ← nrecv(nxtStripA, (m/P)*l, (i+P-1)%P, 1) // recv request of next strip of A
    multiplication(curStripA, stripB, blockC) // C block computation with current strips of A and B
    comWait(idS) // Waiting for send completion
    comWait(idR) // Waiting for recv completion
    exchange(curStripA, nxtStripA) // Array swap of local strips of A for next iteration
  endfor
  multiplication(curStripA, stripB, blockC) // Computation of last block of C for column M of B
endfor
    
```

Performance evaluation

Let's define : t_b = computation time of one **block** of C

t_s = communication time of one **strip** of A

Version *without overlapping* :

- $T_W = P.t_b + 2(P-1).t_s$

Version *with communications overlapping* :

- $T_{Cm} = P.t_b + (P-1).t_s$

Version *with comp/comm overlapping* :

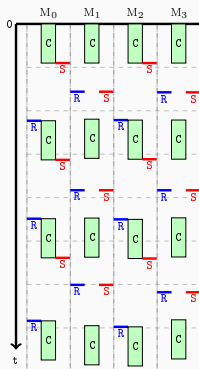
- $T_{CC} = (P-1).max(t_b, t_s) + t_b$

Gains →	Cm	CC
W	$T_W - T_{Cm} = (P-1).t_s$	if $t_s \leq t_b$: $T_W - T_{CC} = 2(P-1).t_s$ if $t_s > t_b$: $T_W - T_{CC} = (P-1).(t_b + t_s)$
Cm		$T_{Cm} - T_{CC} = (P-1).min(t_b, t_s)$

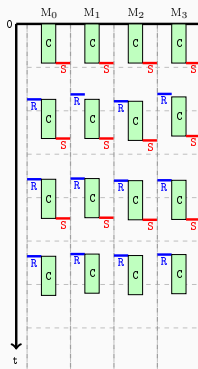
⇒ Overlapping *covers the shortest action* between computations and communications !

Execution times of MPI implementation

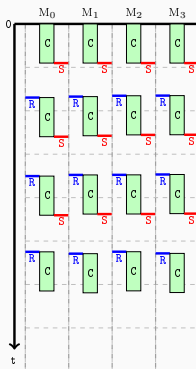
Without overlapping



Non-blocking sendings

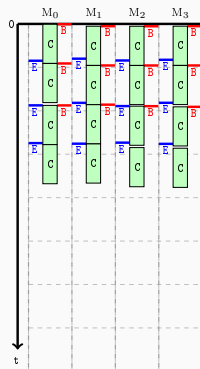


Non-blocking receptions



Comp/comm overlapping

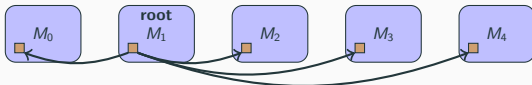
2 threads (OpenMP)



Global communications

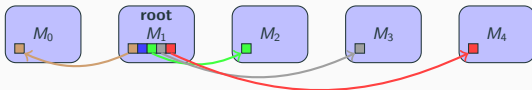
Broadcast : A root process *sends* the *same* data to *all others*

```
bcast(data, size, root)
```



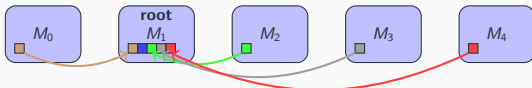
Scattering : A root process *sends different* data to *all others*

```
scatter(dataToSend, size, receivedData, root)
```



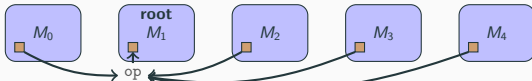
Gathering : A root process *concatenates* data *received* from *all others*

```
gather(dataToSend, sizeSent, receivedData, root)
```



Reduction : A root process *combines* data *received* from *all others* via an *operation*

```
reduction(dataToSend, size, op, resultData, root)
```



Example of the broadcast

```
function bcast(data, size, root)
Begin
  pId ← getProcessId()  // Retrieves the id of the current process
  pNb ← getProcessNb()  // Retrieves the total number of processes

  if pId = root then      // Root process
    // Loop that sends data to the other processes
    for dest from 0 to pNb-1 do
      if dest ≠ root then // Exclusion of the root (already has the data)
        send(data, size, dest, btag) // btag value should be reserved
                                     // for bcasts only
      endif
    endfor
  else                  // Other processes
    // Reception on every process different from root
    recv(data, size, root, btag)
  endif
End
```

Synchronization

Interest :

- Controls the progression between consecutive stages in an algorithm
- After a synchronization, every machine is sure that the others have passed that point too
- Generally required at least at start and end of the algorithm

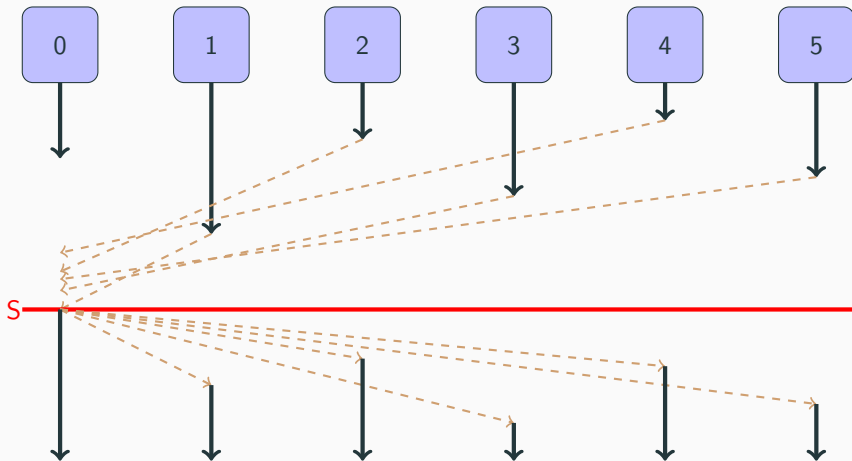
Disadvantages :

- Slows down the global process (waitings)
- Some global communications have a synchronizing effect

Principle :

- One *master* :
 - Waits for a message from all the others
 - Once *all* messages are received, acknowledgment is sent to the others
- Other machines/units :
 - Send a message to the master
 - Wait for the acknowledgment

Execution diagram



Algorithmic schemes for distributed memory systems

Problem decomposition :

- Objective is to extract maximal parallelism with middle/coarse grain
- Possible approaches :
 - *Domain* decomposition (data sub-parts)
 - *Task* decomposition (pipeline or oriented graph)

Data repartition :

- *Duplication* :
 - ☺ Simpler management
 - ☺ Reduction of communication amount → better performance
 - ☹ Restriction of the problem size (loss of one aspect of parallelism)
- *Distribution* :
 - ☺ Better exploitation of resources → larger problems
 - ☹ Generally induces more communications
 - ☹ More difficult management

Summary of distributed memory systems programming

Advantages :

- Access to a much larger computing power :
⇒ *Scalability* allows to treat large size problems
- Quite simple mechanisms to transfer data
- Possibility to *overlap computations/communications*
⇒ Multi-threading may be necessary

Disadvantages :

- Communication *Cost*!
 - Required send/recv correspondence in synchronous and blocking modes
⇒ Risk of *deadlocks*!
 - *Data distribution* may be *tedious*
 - Identifying an efficient distribution is not always straightforward
- ⇒ **Large scale parallelism that must be combined to a lower scale parallelism on each machine to obtain maximal efficiency**

Implementation of parallelism in shared memory systems

Implementation of parallelism in shared memory systems

Main contexts of shared memory :

- Multi-processor machines
- Multi-core processors
- GPUs, Xeon-Phi

Use of *threads* :

- Lightweight processes that :
 - Share the memory space of the parent process
 - Can be executed on any hardware core
- Native programming :
 - Neither very convenient nor always efficient (optimization)

⇒ Specific libraries : *OpenMP*

- Compilation directives : `#pragma omp ...`
- In addition to a sequential code : two versions in one !

Different kinds of multithreadings

There are mainly *two forms* of parallelism in a processor :

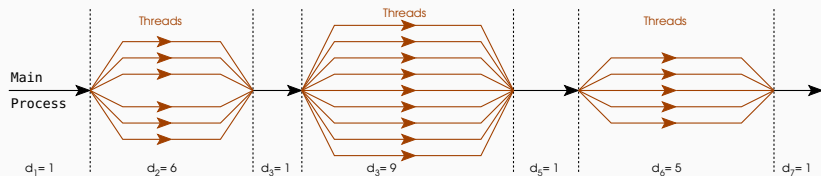
- *Superscalar* : instruction level parallelism (*ILP*) → several ALUs
- *Multithreading* : thread level parallelism (*TLP*)

There are also *several* kinds of *multithreadings* :

- *Interleaved* : sequential execution of different threads
 - *Fine grain* : strict interleaving of the threads (fixed number of cycles)
 - *Coarse grain* : dynamical switch between threads (mem accesses,...)
- *Simultaneous* (SMT) : parallel execution of different threads
- *Multicores* : several cores independently execute different threads

Fundamental principles of OpenMP

Number of *threads* related to the *degree* of each part of the algorithm



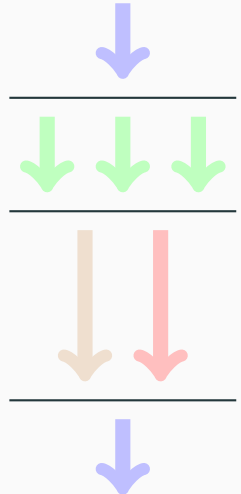
It is possible to start from a sequential algorithm :

- Progressive parallelization (one part at a time)
- Possible validation after each partial parallelization

However, it is sometimes required to *re-organize* the initial algorithm to obtain an efficient parallel version

Syntax and semantics

```
int main()
{
    ...           // Sequential part
    #pragma omp parallel num_threads(3)
    {
        ...       // Creation of 3 threads
        ...       // Code duplication over 3 threads
        #pragma omp sections
        {
            // Work distribution
            #pragma omp section
            {
                ... // Work done by only one thread
            }
            #pragma omp section
            {
                ... // Work done by another thread
            }
        }
    }
    ...           // Return to sequential part
}
```



OpenMP API

Set of functions declared in :

```
#include <omp.h>
```

Useful to :

- Obtain information :

```
int omp_get_num_threads() // number of active threads
int omp_get_thread_num() // number of current thread
double omp_get_wtime() // clock (seconds)
```

- Configure some settings :

```
void omp_set_num_threads(...) // sets the default number of threads
```

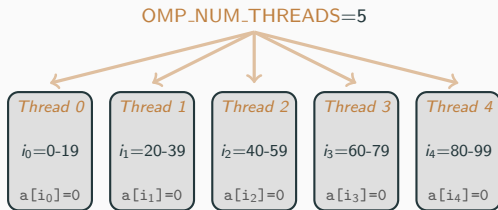
- Manage locks :

```
void omp_init_lock(...) // lock initialization
void omp_set_lock(...) // lock request (blocking)
void omp_unset_lock(...) // unlocking
int omp_test_lock(...) // lock checking (non blocking)
void omp_destroy_lock(...) // lock destruction
```


Loop directive

Distribution of loop iterations over the active threads

```
#pragma omp parallel
{
  ...
  #pragma omp for [clause,...]
  for(i=0; i<100; i++){
    a[i] = 0;
  }
  ...
}
```



The optional clause is used for specifying the variables management :

- `private(var)` :
 - A *private* copy of `var` is built for each thread
 - Specification of initialization and final value (after the loop)
- `shared(var)` :
 - `var` is the *same* variable for *all* threads
 - ⚠ *Be careful* of read/write coherency !!
 - *Default* mode can be changed : `default(none)` / `default(private)`

Static scheduling

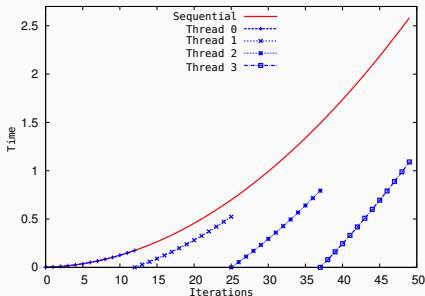
```
#pragma omp for schedule(static, size)
for(i=0; i<50; i++){
    my_computation(i);
}
```

The clause `schedule(static,...)` *distributes* the iterations by *groups* of specified size :

- Default distribution mode
 - When no size is specified, the number of iterations is divided by the number of threads
 - Groups distribution follows the *cyclic order* of the threads
- Efficient when iterations take *similar times*

Otherwise, *imbalances* are possible :

- Iterations with increasing times, executed by 4 cores



Dynamic scheduling

```
#pragma omp for schedule(dynamic, size)
for(i=0; i<50; i++){
    my_computation(i);
}
```

The clause `schedule(dynamic,...)` distributes *on-the-fly* the iterations groups with specified size :

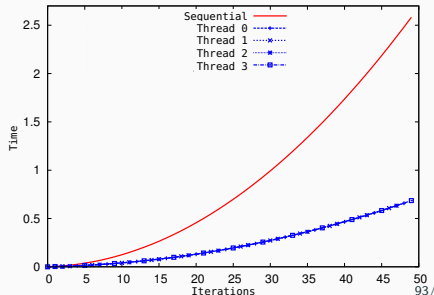
- Default groups size is 1
- Each iterations group is assigned to the *first available thread*
- Overhead due to dynamical groups distribution

- *Inefficient* with similar iterations

→ Required when iterations take *different times*

Implicit balancing :

- Iterations with increasing times, executed by 4 cores



Instructions blocks management

Several directives for managing the execution of instructions blocks :

- `single` :
 - The block is executed by *only one* thread
 - The other threads wait for the end of execution (*synchronization*)
 - Useful to avoid breaking of parallel sections (*expensive*)
- `master` :
 - The block is executed by the *master thread only* (thread 0)
 - No synchronization with the other threads
- `critical` (or `atomic` for a single instruction) :
 - The block is executed by *all* threads, but *one at a time*
 - Useful to manage *mutual exclusion*
- `barrier` :
 - Explicit synchronization between threads
 - Also present at the end of directives `parallel`, `sections`, `single`
 - Useful to ensure coherency of consecutive computations

Examples

```
#pragma omp parallel
{
    int num = omp_get_thread_num();
    int nbT = omp_get_num_threads();

    printf("Thread %d among %d\n", num, nbT);

    // Sequential part
    #pragma omp single
    {
        printf("Thread %d alone\n", num);
    }

    printf("Thread %d among %d\n", num, nbT);
}
```

```
int nbT = 0; // Number of threads

#pragma omp parallel
{
    #pragma omp critical
    {
        nbT++;
    }
}

printf("%d threads\n", nbT);
```

```
#pragma omp parallel
{
    int num = omp_get_thread_num();
    int nbT = omp_get_num_threads();

    printf("Thread %d among %d\n", num, nbT);

    // Sequential part
    #pragma omp master
    {
        printf("Thread %d alone\n", num);
    }

    printf("Thread %d among %d\n", num, nbT);
}
```

```
#pragma omp parallel
{
    #pragma omp sections
    {
        ...
    }

    #pragma omp barrier

    #pragma omp for
    for(int i=0; i<N; ++i){
        ...
    }
}
```

Reduction and tasks

Reduction clause :

- Can be applied to directives : `parallel`, `sections`, `for`
- 1st parameter : reduction *operation* (+, ×, min, max, &, |,...)
- 2nd parameter : shared *variables list* upon which reduction is applied

Dynamical creation of tasks : `#pragma omp task`

- The associated block can be executed by the parent thread or any other thread available in the current parallel section
- Useful to distribute *dynamical tasks*
- Commonly, a single thread creates the tasks :
 - Use of the `task` directive in a `single` section
 - Initialization → tasks creation → potential wait (`taskwait`)

Always better to use the `for` directive whenever it is possible

Examples

```
int accu = 0;

#pragma omp parallel
{
    #pragma omp for reduction(+:accu)
    for(int i=0; i<N; ++i){
        accu++;
    }
}

printf("Accu = %d\n", accu);
```

```
#pragma omp parallel
{
    #pragma omp single
    {
        printf("Fibonacci(%d) = %d\n", fibo(n));
    }
}
```

```
int fibo(int n)
{
    if ( n == 0 || n == 1 ) { return n; }

    int fnm1, fnm2;

    #pragma omp task shared(fnm1)
    fnm1 = fibo(n-1);

    #pragma omp task shared(fnm2)
    fnm2 = fibo(n-2);

    #pragma omp taskwait

    return fnm1 + fnm2;
}
```

Conditional parallelism and desynchronization

Conditional parallelism : `#pragma omp parallel if(condition)`

- Creates the threads *only if* the condition is true
Otherwise, the block is executed by the main process → *sequential*
- Creating threads is not free !
 - ⇒ Not efficient when too few computations to distribute
 - The risk is to have $\text{gain}(\text{parallelism}) < \text{overhead}(\text{parallelism})$
- The condition is generally related to the number of data to process
- Accurate setting requires good knowledge of the machine
 - Theoretical performance, reference benchmarks, runtime measures

Desynchronization : `nowait`

- *Optional* clause for some directives
- *Suppression* of implicit *synchronizations* at end of blocks
- Useful to avoid superfluous blocking of threads

Examples

```
#pragma omp parallel if(N > 10000)
{
    #pragma omp for
    for(int i=0; i<N; ++i){
        tab[i] = f(i);
    }
}
```

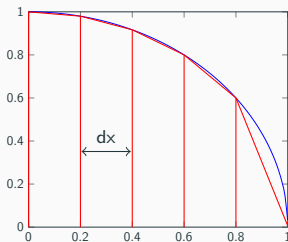
```
#pragma omp parallel
{
    #pragma omp for nowait
    for(int i=0; i<N; ++i){
        a[i] = f(i);
    }

    #pragma omp for
    for(int i=0; i<N; ++i){
        b[i] = g(i); // no dependence with a[]
    }
}
```

Example of π approximation

Approximation of π by the *trapezoid rule* :

- nbTr is the number of trapezoids
- $dx = 1/\text{nbTr}$ is the width of the trapezoids



```
pi = 0.5; // (f(0.0) + f(1.0)) / 2.0
#pragma omp parallel for private(x) reduction(+:pi)
for(i=1; i<nbTr-1; i++){
    x = dx * i;
    pi += sqrt(1.0 - x * x);
}
pi = 4.0 * dx * pi;
printf("The approximation of PI is : %f\n", pi);
```

Summary of shared memory systems programming

Advantages :

- Fast deduction of a parallel version from a sequential one
 - ⇒ Efficient with *regular* data and *independent* computations
 - May require *computation reorganization* to extract parallelism
- Possible to design parallel algorithms from scratch
- *Progressive development* with *validation* at each stage
- Can be used on many hardware :
 - Multi-core machines : stations, laptops, smartphones, pads,...

Disadvantages :

- *Concurrent accesses* may be difficult to handle and may limit performance
- *Number of units* is generally limited to a few tens
- *Memory* speed and amount may be a limiting factor

Larger scale parallelism may be necessary :

- Distributed memory systems, nested/hierarchical parallelism

Implementation of parallelism in distributed memory systems

Implementation of parallelism in distributed memory systems

Specific communication libraries :

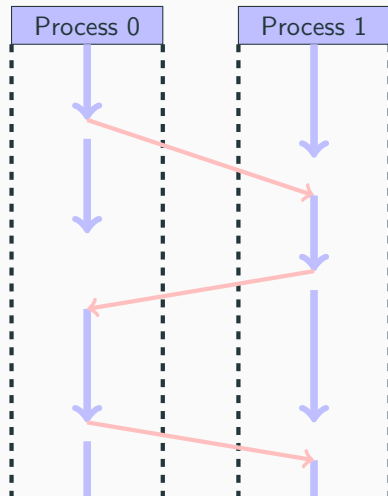
- *Explicit communications* : MPI, PVM
 - API providing functions to send/recv messages
- *Implicit communications* :
 - Environments based on *RPC* (Remote Procedure Call)
 - Call of a function potentially located on another machine
 - Function parameters are sent via a message
 - Function results are received via a message
 - Environments based on *virtual shared memory* :
 - All the machines see a *single shared memory* (virtual)
 - Messages are automatically generated when accessing non-local data
 - Simple reuse of SM codes but performance may not be good

Fundamental principle of MPI

A set of independent processes exchange data through explicit messages

SPMD scheme :

- A *same program* is executed by all the processes
- *Different execution paths* distinguish the work of each unit :
 - Computations, communications
- *Correspondence between send/recv calls* is mandatory in synchronous and blocking modes
- Sometimes, one process plays a central role :
 - *Master-Workers* scheme
 - I/O interface



Main functions of the C version of MPI

API inclusion :

- `#include <mpi.h>`

Initialization and finalization :

- `MPI_Init(&argc, &argv);` starts the communication environment
- `MPI_Finalize();` stops the communication environment
- *All* communications must be done between these two calls !

Information :

- `MPI_Comm_size(MPI_COMM_WORLD, &nbP);` number of processes
- `MPI_Comm_rank(MPI_COMM_WORLD, &numP);` process ID
- `MPI_Wtime();` time measure (s)

Point to point communication :

- Synchronous mode, blocking and non blocking asynchronous modes

Synchronous :

`MPI_Ssend()`

`MPI_Recv()`

Blocking :

`MPI_Send()`

`MPI_Recv()`

Non blocking :

`MPI_Isend()`

`MPI_Irecv()`

Basic send/receive functions

Send : `MPI_Send(data, nb, type, dest, tag, comm)`

- `data` : data array to send
- `nb` : number of data to send
- `type` : type of data to send (predefined constants)
 - `MPI_CHAR`, `MPI_BYTE`, `MPI_INT`, `MPI_DOUBLE`, ...
- `dest` : destination ID in communication group `comm`
- `tag` : message tag for filtering on the receiver side
- `comm` : communication group (`MPI_COMM_WORLD` for *all* processes)

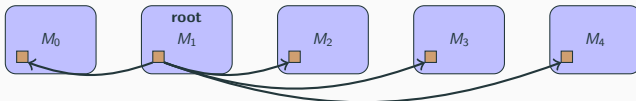
Receive : `MPI_Recv(data, nb, type, src, tag, comm, state)`

- `data` : data array in which to store received data
 - ⇒ Must be *allocated before* the reception with *sufficient size*
- `src` : ID of the message source in the communication group `comm`
- `tag` : message filtering according to the tag
- `state` : information about the reception state (`NULL` when not used)

One-to-all global communications

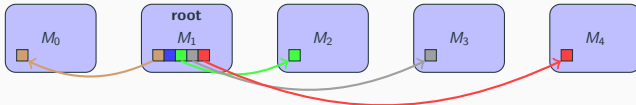
Broadcast : `MPI_Bcast(data, nb, type, root, comm)`

- data : data *sent* by root and *received* by the others
- root : ID of the process that broadcasts to the others
- Different actions over the processes



Scatter : `MPI_Scatter(dataS, nbS, typeS, dataR, nbR, typeR, root, comm)`

- dataS : array of data to scatter (root included in the destinations)
- nbS : number of data to scatter per process
- dataR : array to store the received data on each process
- root : ID of the process that scatters the data



```
void MPI_Bcast(void *data, int nb, MPI_Datatype type,
               int root, MPI_Comm comm)
{
    int i, idP, nbP; // Counter, process ID and number of processes
    MPI_Status state; // Reception state

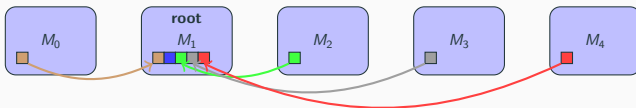
    MPI_Comm_rank(MPI_COMM_WORLD, &idP); // Process ID
    MPI_Comm_size(MPI_COMM_WORLD, &nbP); // Number of processes

    if(idP == root){ // Root process
        // Loop that sends data to the other processes
        for(i=0; i<nbP; ++i){
            if(i != root){ // Exclusion of the root (already has the data)
                MPI_Send(data, nb, type, i, 1, comm);
            }
        }
    }else{ // Other processes
        // Reception on every process different from root
        MPI_Recv(data, nb, type, root, 1, comm, &state);
    }
}
```

All-to-one global communications

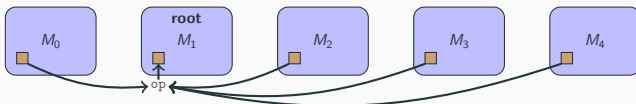
Gather : `MPI_Gather(dataS, nbS, typeS, dataR, nbR, typeR, root, comm)`

- `dataR` : must be large enough to contain *all* the data to receive
- `nbR` : number of data to receive from *each* process



Reduction : `MPI_Reduce(dataS, dataR, nb, type, op, root, comm)`

- `dataS` : may be an *array* $\rightarrow$ result is an array (*compent-wise* `op`)
- `op` : predefined constants : `MPI_MAX`, `MPI_SUM`, `MPI_PROD`, ...



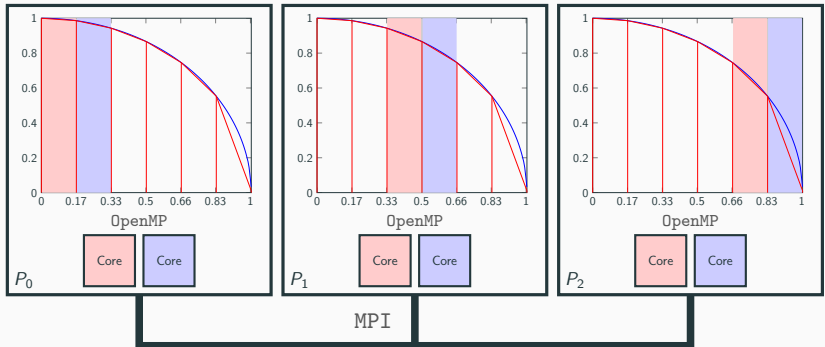
Computation of π by trapezoid integration

Version with *two levels of parallelism* :

→ Set of machines (*distributed* memory) : MPI

→ Set of cores (*shared* memory) : OpenMP

Example of 3 machines with 2 cores



Computation of π by trapezoid integration

Version with *two levels of parallelism* :

→ Set of machines (*distributed* memory) : MPI

→ Set of cores (*shared* memory) : OpenMP

```
MPI_Comm_rank(MPI_COMM_WORLD, &idP); // Process ID
MPI_Comm_size(MPI_COMM_WORLD, &nbP); // Number of processes
piLoc = 0.0; // Partial value of pi computed by idP

// Broadcast of the total number of trapezoids
MPI_Bcast(&nbTr, 1, MPI_INT, 0, MPI_COMM_WORLD);

dx = 1.0 / nbTr; // Deduction of the trapezoids width
nbTrLoc = nbTr / nbP; // Simple case where division is exact

// Partial integration of pi by process idP with nbT threads
#pragma omp parallel for private(x) reduction(+:piLoc) num_threads(nbT)
for(i=idP*nbTrLoc; i<(idP+1)*nbTrLoc; i++){
    x = dx * i;
    piLoc += sqrt(1.0 - x * x);
}

// Global sum of the partial integrations by process 0
MPI_Reduce(&piLoc, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

if(idP == 0){ // Computation and display of final result by process 0
    pi = 4.0 * dx * (pi + 0.5); // Final result
    printf("The approximation of PI is : %lf\n", pi); // Display
}
```

Non blocking send/receive functions

Send : `MPI_Isend(data, nb, type, dest, tag, comm, request)`

- Instantaneous return of the function → *delegation* of work
- Parameters similar to `MPI_Send` save for the last one
- `request` : pointer to a variable of type `MPI_Request` (comm ID)

Receive : `MPI_Irecv(data, nb, type, src, tag, comm, request)`

- Similar to the sending but on the receiver side

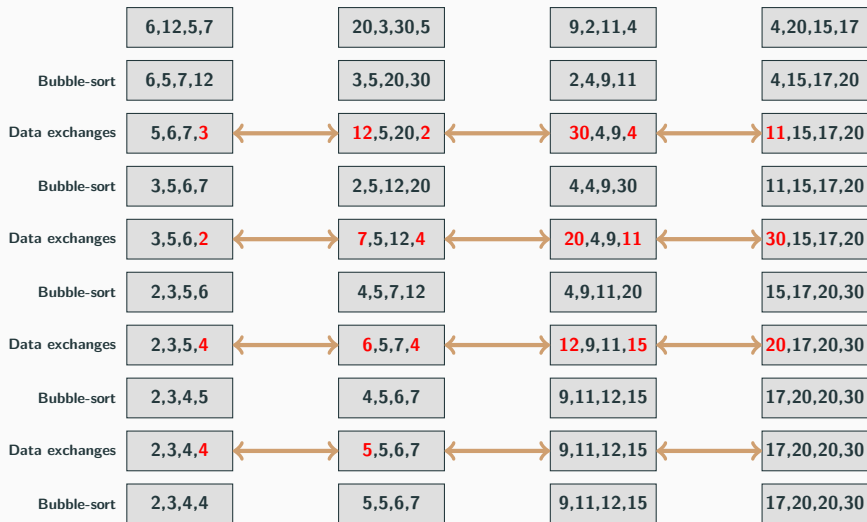
Test : `MPI_Test(request, result, state)`

- `request` : pointer to the communication request to check
- `result` : pointer to the integer result (1 if over, 0 otherwise)
- `state` : pointer to information over the communication

Waiting : `MPI_Wait(request, state)`

- Blocks the process until the communication is done

Example of double bubble sort

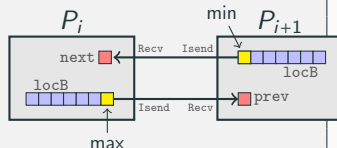


Parallel algorithm for double bubble sort

```

MPI_Request reqF, reqL; // Send requests of first and last local elements
char sorted = 0;        // Boolean indicating if the buffer is globally sorted
locS = SIZE/nbP;        // Size of local buffer locB
// Data distribution of initial buffer buf to the processes
MPI_Scatter(buf, locS, MPI_INT, locB, locS, MPI_INT, 0, MPI_COMM_WORLD);
while(!sorted){ // Succession of buffer parsings with bubbles moves
    // until the buffer is globally sorted
    for(j=0; j<locS-1; ++j){ // Left -> right parsing
        if(locB[j] > locB[j+1]){
            ... // Cells swapping
        }
    } // After this loop, locB[locS-1] contains the local max
    for(j=locS-2; j>0; --j){ // Right -> left parsing
        if(locB[j] > locB[j+1]){
            ... // Cells swapping
        }
    } // After this loop, locB[0] contains the local min
    if(idP > 0){ // Sending 1st elt (min) to left and receiving last elt (max) from left
        MPI_Isend(&locB[0], 1, MPI_INT, idP-1, 1, MPI_COMM_WORLD, &reqF);
        MPI_Recv(&prev, 1, MPI_INT, idP-1, 1, MPI_COMM_WORLD, &state);
        MPI_Wait(&reqF, &state); // Waiting for the sending of 1st elt
        if(prev > locB[0]) locB[0] = prev; // Potential replacement of the 1st elt
    }
    if(idP < nbP - 1){ // Sending last elt (max) to right and receiving first elt (min) from right
        MPI_Isend(&locB[locS-1], 1, MPI_INT, idP+1, 1, MPI_COMM_WORLD, &reqL);
        MPI_Recv(&next, 1, MPI_INT, idP+1, 1, MPI_COMM_WORLD, &state);
        MPI_Wait(&reqL, &state); // Waiting for the sending of last elt
        if(next < locB[locS-1]) locB[locS-1] = next; // Potential replacement of the last elt
    }
    sorted = checkSorted(); // Checks if the buffer is globally sorted
}

```



Main functions of the Python version of MPI

API import : `from mpi4py import MPI`

Initialization and finalization :

- Supported via MPI module import from mpi4py

Communication groups :

- Global : `comm = MPI.COMM_WORLD`

Information :

- Number of processes : `comm.Get_size()`
- Process ID : `comm.Get_rank()`
- Time measure (s) : `MPI.Wtime()`

Point to point communication :

- Both blocking and non blocking asynchronous modes are available

Blocking : `comm.send()` `comm.recv()`

Non blocking : `comm.isend()` `comm.irecv()`

- Distinction between communication of *Python objects* and *arrays*
Objects version (`comm.send`) and arrays version (`comm.Send`)

Standard send/recv functions

Send : `comm.send(data, dest=..., tag=...)`

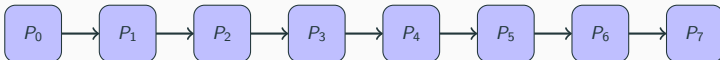
- `comm` : current *communicator* object
- `data` : Python *object* to send
- `dest` : ID of *destination* process in group `comm`
- `tag` : integer used for *filtering* when receiving

Receive : `data = comm.recv(source=..., tag=...)`

- `comm` : current *communicator* object
- `source` : ID of *source* process in group `comm`
- `tag` : *filter* upon the specified tag
- `data` : variable storing the received Python *object*

Simple token example

Logical organization of processes in *line* : example with 8 processes



```
comm = MPI.COMM_WORLD # Communicator definition
rank = comm.Get_rank() # Process ID
size = comm.Get_size() # Total number of processes

if(rank > 0):
    # Processes that have a predecessor
    # WAIT for a message from it (BLOCKING coms)
    val = comm.recv(source=rank-1, tag=10)
    # Display of received value
    print(rank, ":", val)
else:
    print()

# Every process prints its ID and the group size
print("Hello from %d among %d" % (rank, size))

if(rank < size-1):
    # Processes that have a successor send it their ID
    # --> Process 0 unlocks the process 1, and so on...
    comm.send(rank, dest=rank+1, tag=10)
    # The tag must correspond to the one in the recv
```

Broadcast example

```
function bcast(data, size, root)
Begin
  pId ← getProcessId()  // Id of current process
  pNb ← getProcessNb()  // total number of processes

  if pId = root then    // Root process
    // Loop that sends data to the other processes
    for dest from 0 to pNb-1 do
      if dest ≠ root then // Exclusion of the root
                           // (already has the data)
        send(data, size, dest, btag)
      endif
    endfor
  else                  // Other processes
    // Reception on every process different from root
    rcv(data, size, root, btag)
  endif
End
```

Broadcast algorithm

Python broadcast

```
def bcast(data, root, comm):

  pId = comm.Get_rank() # Process ID
  pNb = comm.Get_size() # Total number of processes

  if pId == root : # Root process

    # Loop that sends data to the other processes
    for dst in range(pNb):
      if dst != root : # Exclusion of the root
                        # (already has the data)
        comm.send(data, dest=dst, tag=10)

  else:                # Other processes

    # Reception on every process different from root
    data = comm.rcv(source=root, tag=10)
    print("Process", pId, "receives", data, "from", root)
```

Workload balancing

Workload balancing

In a parallel system, *homogeneous distribution* of tasks or data is *not always efficient*

Some *imbalances* may come from :

- *Heterogeneity between machines* :
 - Different speeds or workloads of the computing units
- *Heterogeneity between computations* :
 - Tasks with different amounts of computation
 - Tasks generated dynamically during the execution of the algorithm
- *Heterogeneity between data* :
 - Dynamic evolution of the number of data to process
 - The process of one datum may generate other data to process

⇒ The objective of *load balancing* is to distribute the work so that all computing units *finish at the same time*

Simple example

Let's consider :

- Three units P_0 , P_1 et P_2 with the following speeds :
 - $V_0 = V_1 = V$ and $V_2 = 2V$ op/s
- A parallel computation C whose cost is α operations per datum

Homogeneous distribution of N data other three units :

- $T(C(\frac{N}{3}, P_0)) = T(C(\frac{N}{3}, P_1)) = \frac{\alpha \cdot N}{3V}$ and $T(C(\frac{N}{3}, P_2)) = \frac{\alpha \cdot N}{6V}$

$$\Rightarrow T_3(N) = \frac{\alpha \cdot N}{3V}$$

Can we do better ?

- **Yes**, by giving more data to P_2
- We need to satisfy :

$$T(C(N_0), P_0) = T(C(N_1), P_1) = T(C(N_2), P_2)$$

$$\Rightarrow \frac{N_0}{V} = \frac{N_1}{V} = \frac{N_2}{2V} \Rightarrow N_0 = N_1 \text{ and } N_2 = 2N_0 = 2N_1 \Rightarrow T_3(N) = \frac{\alpha \cdot N}{4V}$$

Static allocation of identical tasks

We have N independent tasks, with same cost, to distribute over M units

The *static distribution* consists in :

- Evaluating the *respective speeds* of the units : V_i op/s
- Deducing the *total computing power* of the system : $P = \sum_{j=0}^{M-1} V_j$
- Deducing the *relative power* of each unit : $P_i = \frac{V_i}{P}$
- And the *load* C_i (nb of tasks) to give to each unit according to its P_i

$$\Rightarrow C_i = P_i \cdot N$$

```
P ← sum(V, 0, M-1)    // Sum of the values of array V
for i from 0 to M-1 do // Computation of the loads of the units
  C[i] ← floor(V[i] * N / P)
endfor
while sum(C, 0, M-1) < N do // Distribution of the remaining tasks
  k ← argmin_j((C[j] + 1) / V[j]) // Similar to the list algorithm
  C[k] ← C[k] + 1
endwhile
```

⇒ Algorithm updating the optimal distribution when N increases

Dynamic load balancing with a queue

Use of a *FIFO queue* :

- To store the tasks to do
- To store the data to process

Principle :

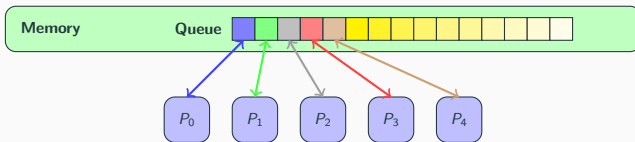
- The units take their next task/data from the queue
- At the end of a task, new tasks or data may be added to the queue

Termination :

- The queue must be empty *and* all units must be inactive
- ⇒ This queue mechanism implements an *implicit balancing* of the workload over the units

Queue in shared memory

Relatively simple and efficient strategy :



But :

- All the units access the *same* queue!!

⇒ We need to ensure the *accesses coherency* :

- *Mutual exclusion* is required for each access to the queue
- Any queue modification must be placed in a *critical section*
The queue *modifications* come from :

- Removing one task/data
- Inserting a new task/data

Read accesses are required to :

- Check if the queue is not empty
- Get the next task/data

Queue management algorithm

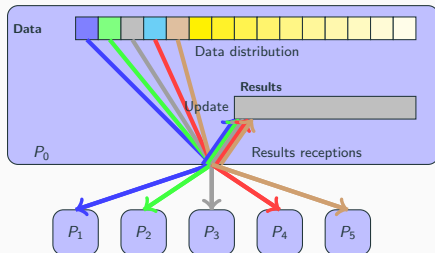
```
for p from 0 to n-1 do in parallel
  if p = 0 then // Only one unit initializes the queue and lock
    queue ← ConstructQueue(tasksList)
    initLock(v)
    nbActive ← 0
  endif
  barrier() // Prevents other units to access data before their initialization
  finishedp ← False // Indicates if unit p must stop the work loop
  taskp ← Nil // Task (or data) to process
  while !finishedp do // Work loop of unit p
    lock(v) // Entering in critical section to access the queue
    if ¬isEmpty(queue) then // It there is at least one remaining task...
      taskp ← unqueue(queue) // ...one task is retrieved
      nbActive ← nbActive + 1 // ...and the current unit is counted as active
    else // Else...
      if nbActive = 0 then // ...we check that there is no more running task
        finishedp ← True // ...in this case, unit p stops
      endif
    endif
    unlock(v) // Leaving the critical section
    if taskp ≠ Nil then // If a task is retrieved
      execute(taskp) // ...it is executed
      lock(v)
      nbActive ← nbActive - 1 // ...and unit p is removed from the active units
      unlock(v)
    endif
  endwhile
endfor
```

Queue in distributed memory

Principle of the *master/worker* :

- One *master* unit controls the queue
- The other units :
 - Ask tasks to the master
 - Return their results

⇒ *Centralized schema*



Advantages :

- Very *simple* to implement
- *Unique* access to the queue

Drawbacks :

- The master unit may be a *bottleneck* :
 - Limits the number of workers
 - Limits the grain of tasks to distribute

Important remarks on using tasks/data queues

It is important to avoid *starvation* phenomena :

- The master unit must give work to *all* workers *before* a new demand
- The time of a task must be large enough

Fault tolerance can also be added :

- *Redistribute a task* if the result does not come fast enough (network or machine failure,...)

Competition between machine if they have different speeds :

- Distribute a *same task* to *several units*
- ⇒ The result comes from the fastest unit
- ⇒ Other tasks are sent to the other units (previous work is canceled)

Dynamic load balancing by work redistributions

The tasks are *regularly redistributed* between the units :

- Avoids the centralized queue
- Takes the evolution of the system into account during the execution

Principle :

- Initial distribution of the work (static)
- Between main steps or at regular time intervals :
 - Evaluates the load/performance of every unit
 - Computes a new balanced work distribution
 - Redistributes the data/tasks

⚠ !! Global redistributions are expensive in distributed memory !! ⚠

Requires an appropriate setting of the *redistributions frequency* :

- If *too high* : the redistributions cost may be greater than the balancing gain
- If *too low* : risk of extended imbalance

Local balancing of work

Machines are connected via a *logical interconnection graph* :

- A unit with not enough work asks some to its neighbors
- ⇒ The *local work load* on a unit must be *divisible*
- Choice of the parallel grain : several tasks per unit

Advantages :

- Load transfers only between neighbors
- ⇒ *Local communications* are favored (faster)

Rules to follow to obtain an efficient balancing :

- Avoid *ping-pong* effect
 - Continuous exchanges of load between two units
 - Occurs when the imbalance between the units is close to the threshold
- Avoid *starvation* by sending too much work to neighbors
- Limit the number of load transfers
 - Prevents network overloading and slow down of computations

Conclusion

Conclusion

Parallelism provides :

- A *gain in execution time*
- The possibility to process *large size problems*

There are several kinds of parallelism :

- Data, tasks, pipeline
- Generally combined under the form of a *hierarchical composition*

The main obstacles are :

- The *dependencies* between computations
- The *concurrent accesses* in shared memory
- The *communications cost* in distributed memory
- The efficient management of the different levels of nested parallelism

The efficient use of super-computers is still a **challenge** !

Actual software rarely reach theoretical peak power of hardware specs